

SOLIDWORKS API (.NET VBA C#) — Examples (1/50): Access and Release Access to Edges, Examp- Add .NET Controls to SOLIDWORKS Using an

SOLIDWORKS API Documentation (converted from CHM)

Access Edges on Rip Feature Example (C#)

This example shows how to access edges on a rip feature.

```
//-----
// Preconditions:
// 1. Verify that the specified model document exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Creates Shell1 and Rip1 features.
// 3. Inspect the FeatureManager design tree, the graphics area, and
//    the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace InsertRipFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            RipFeatureData swRip = default(RipFeatureData);
            object[] vEdge = null;
            Edge swEdge = default(Edge);
            Entity swEnt = default(Entity);
            int i = 0;
            int lRipDirection = 0;
            bool bRet = false;
            int longstatus = 0;
            int longwarnings = 0;

            lRipDirection = 64;

            swModel = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\box.sldprt", 1, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("box", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            bRet = swModel.Extension.SelectByID("", "FACE", -0.0566885410894997, 0.02999999999999159, 0.0200993374069753, false, 1,
            swModel.InsertFeatureShell(0.01, false);

            bRet = swModel.Extension.SelectByID("", "EDGE", 0.0441585455038194, 0.0151990980971277, 0.0459121462268968, true, lRipD
            swModel.InsertRip(0.0001);

            bRet = swModel.Extension.SelectByID("Rip1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);

            // Get rip feature data
            swRip = (RipFeatureData)swFeat.GetDefinition();

            Debug.Print("Gap: " + swRip.Gap * 1000.0 + " mm");

            bRet = swRip.AccessSelections(swModel, null);

            Debug.Print("Number of edges: " + swRip.GetEdgesCount());

            swModel.ClearSelection2(true);

            vEdge = (object[])swRip.Edges;

            for (i = 0; i <= vEdge.GetUpperBound(0); i++)
            {
                swEdge = (Edge)vEdge[i];
                swEnt = (Entity)swEdge;

                Debug.Print("Direction of rip for edge (0=current, 1=other, 2=both): " + swRip.GetDirection(swEdge));

                bRet = swEnt.Select4(true, null);
            }

            swRip.ReleaseSelectionAccess();
        }
    }
}
```

```
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}
```

Access Edges on Rip Feature Example (VBA)

This example shows how to access edges on a rip feature.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Creates Shell1 and Rip1 features.
' 3. Inspect the FeatureManager design tree, the graphics area, and
'    the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swRip As SldWorks.RipFeatureData
    Dim vEdge As Variant
    Dim swEdge As SldWorks.Edge
    Dim swEnt As SldWorks.Entity
    Dim i As Long
    Dim lRipDirection As Long
    Dim bRet As Boolean
    Dim longstatus As Long, longwarnings As Long

    Set swApp = Application.SldWorks
    lRipDirection = 64
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt", 1, 0, "",
longstatus, longwarnings)
    swApp.ActivateDoc2 "box", False, longstatus
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    bRet = swModel.Extension.SelectByID2("", "FACE", -5.66885410894997E-02, 2.99999999999159E-02, 2.00993374069753E-02, False, 1,
Nothing, 0)
    swModel.InsertFeatureShell 0.01, False

    bRet = swModel.Extension.SelectByID2("", "EDGE", 4.41585455038194E-02, 1.51990980971277E-02, 4.59121462268968E-02, True,
lRipDirection, Nothing, 0)
    swModel.InsertRip 0.0001

    bRet = swModel.Extension.SelectByID2("Rip1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)

    ' Get rip feature data
    Set swRip = swFeat.GetDefinition

    Debug.Print "Gap: " & swRip.Gap * 1000# & " mm"
    bRet = swRip.AccessSelections(swModel, Nothing)

    Debug.Print "Number of edges: " & swRip.GetEdgesCount
    swModel.ClearSelection2 (True)
    vEdge = swRip.Edges
    For i = 0 To UBound(vEdge)
        Set swEdge = vEdge(i)
        Set swEnt = swEdge
        Debug.Print "Direction of rip for edge (0=current, 1=other, 2=both): " & swRip.GetDirection(swEdge)
        bRet = swEnt.Select4(True, Nothing)
    Next i
    swRip.ReleaseSelectionAccess
End Sub

```

Access Edges on Rip Feature Example (VB.NET)

This example shows how to access edges on a rip feature.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Creates Shell1 and Rip1 features.
' 3. Inspect the FeatureManager design tree, the graphics area, and
'    the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swRip As RipFeatureData
        Dim vEdge As Object
        Dim swEdge As Edge
        Dim swEnt As Entity
        Dim i As Integer
        Dim lRipDirection As Integer
        Dim bRet As Boolean
        Dim longstatus As Integer, longwarnings As Integer

```

```
        lRipDirection = 64
```

```

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\box.sldprt", 1, 0, "", longstatus, longwarnings)
        swApp.ActivateDoc2("box", False, longstatus)
        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

```

```

        bRet = swModel.Extension.SelectByID2("", "FACE", -0.0566885410894997, 0.029999999999999159, 0.0200993374069753, False, 1, Notf
swModel.InsertFeatureShell(0.01, False)

```

```

        bRet = swModel.Extension.SelectByID2("", "EDGE", 0.0441585455038194, 0.0151990980971277, 0.0459121462268968, True, lRipDirec
swModel.InsertRip(0.0001)

```

```

        bRet = swModel.Extension.SelectByID2("Rip1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeat = swSelMgr.GetSelectedObject6(1, -1)

```

```

        ' Get rip feature data
        swRip = swFeat.GetDefinition

```

```
        Debug.Print("Gap: " & swRip.Gap * 1000.0# & " mm")
```

```
        bRet = swRip.AccessSelections(swModel, Nothing)
```

```
        Debug.Print("Number of edges: " & swRip.GetEdgesCount)
```

```
        swModel.ClearSelection2(True)
```

```
        vEdge = swRip.Edges
```

```

        For i = 0 To UBound(vEdge)
            swEdge = vEdge(i)
            swEnt = swEdge

```

```
            Debug.Print("Direction of rip for edge (0=current, 1=other, 2=both): " & swRip.GetDirection(swEdge))
```

```
            bRet = swEnt.Select4(True, Nothing)
```

```
        Next i
```

```
        swRip.ReleaseSelectionAccess()
```

```
End Sub
```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Access Assembly Example (C++ COM)

This example shows how to open, explode and collapse, and create a drawing of an assembly. It also shows how to traverse the FeatureManager design tree, get a component, and suppress and resolve that component. This example uses CComPtr Smart_Pointers.htm and CComBSTR smart types.

DemoAssemblies.cpp: application code

```

////////////////////////////////////
//
// Preconditions: ujoint.sldasm exists in the specified folder and
//              contains a component named bracket-1.
//
//
// Postconditions: None
//
// NOTE: Scroll down see the code for the stdafx.h header file.
//
////////////////////////////////////

#include "stdafx.h"

VARIANT_BOOL retVal = VARIANT_FALSE;
HRESULT hres = NOERROR;

//Function prototypes
void OpenAssembly(ISldWorks* swApp, IModelDoc2** swModel);
void TraverseFeatureManagerDesignTree(IModelDoc2* swModel, IComponent2** swComponent);
void SuppressSelectedComponent(IComponent2* swComponent, ISldWorks* swApp);
void ResolveSelectedComponent(IComponent2* swComponent, ISldWorks* swApp);
void ExplodeAssembly(IModelDoc2* swModel, IAssemblyDoc** swAssemblyDoc);
void CollapseAssembly(IModelDoc2* swModel, IAssemblyDoc* swAssemblyDoc);
void CreateDrawingOfAssembly(ISldWorks* swApp, IModelDoc2* swModel);
void CloseDocuments(ISldWorks* swApp);

using namespace std;

int _tmain()
{
    //Initialize COM
    CoInitialize(NULL);

    //Use ATL smart pointers
    CComPtr<ISldWorks> swApp;
    CComPtr<IModelDoc2> swModel;
    CComPtr<IComponent2> swComponent;
    CComPtr<IAssemblyDoc> swAssemblyDoc;

    bool bDone;

    do
    {
        try
        {
            //Open or attach to the currently running instance of the sldworks.exe
            //COM server on your local computer; throw an exception if
            //SOLIDWORKS Design is not opened or attached to
            hres = swApp.CoCreateInstance(__uuidof(SldWorks), NULL, CLSCTX_LOCAL_SERVER);

            if (hres != S_OK)
                throw 0;

            bDone = true;

            IModelDoc2* swModel = NULL;

            //Open assembly
            OpenAssembly(swApp, &swModel);
            IComponent2* swComponent = NULL;

```

```

//Traverse FeatureManager design tree to get specified feature
TraverseFeatureManagerDesignTree(swModel, &swComponent);

//Suppress selected component if it is resolved
SuppressSelectedComponent(swComponent, swApp);

//Resolve selected component if it is suppressed
ResolveSelectedComponent(swComponent, swApp);
IAssemblyDoc* swAssemblyDoc = NULL;

//Explode assembly
ExplodeAssembly(swModel, &swAssemblyDoc);

//Collapse exploded assembly
CollapseAssembly(swModel, swAssemblyDoc);

//Create drawing of assembly
CreateDrawingOfAssembly(swApp, swModel);

//Close documents
CloseDocuments(swApp);

}
//Catch the exception and tell the user that SOLIDWORKS Design is not running
catch (int)
{
    cout << "Error starting or attaching to a SOLIDWORKS Design session." << endl;

    //Release COM references
    swApp = NULL;
    swModel = NULL;
    swComponent = NULL;
    swAssemblyDoc = NULL;

    //Uninitialize COM
    CoUninitialize();

    return 1;
}
}
while(!bDone);

//Release COM references
swApp = NULL;
swModel = NULL;
swComponent = NULL;
swAssemblyDoc = NULL;

//Uninitialize COM
CoUninitialize();

return 0;
}
void OpenAssembly(ISldWorks* swApp, IModelDoc2** swModel)
//Open assembly
{
    CComBSTR sFileName(L"c:\\test\\universaljoint\\ujoint.sldasm");
    CComBSTR sDefaultConfiguration(L"Default");

    long fileerror, filewarning;

    IModelDoc2* swModelAssembly;
    hres = swApp->OpenDoc6(sFileName, swDocASSEMBLY, swOpenDocOptions_Silent, sDefaultConfiguration, &fileerror, &filewarning, &swModelAssembly);
    *swModel = swModelAssembly; //Set the value of the input argument equal to address of the interface
}

void TraverseFeatureManagerDesignTree(IModelDoc2* swModel, IComponent2** swComponent)
// Traverse FeatureManager design tree to get the
// specified feature in FeatureManager design tree
{
    //Use ATL smart pointers

```

```

CComPtr<IFeature> swFeature;
CComPtr<ISelectionMgr> swSelMgr;

CComBSTR sGetFeatureName(L"");
CComBSTR sFeatureName(L"bracket-1");

bool bFoundComponents = false;
hres = swModel->IFirstFeature(&swFeature);
// If the name of the feature is "bracket-1"
// then select the feature
do
{
    hres = swFeature->get_Name(&sGetFeatureName);
    if (sGetFeatureName == sFeatureName)
    {
        hres = swFeature->Select2(true, 1, &retVal);
        hres = swModel->get_ISelectionManager(&swSelMgr);
        struct IDispatch *pComponentDisp;
        hres = swSelMgr->GetSelectedObject6(1, -1, &pComponentDisp);
        IComponent2* swSelectedComponent;
        hres = pComponentDisp->QueryInterface(__uuidof(IComponent2), reinterpret_cast<void**>(&swSelectedComponent));
        pComponentDisp->Release();
        *swComponent = swSelectedComponent; //Set the value of the input argument equal to address of the interface
        bFoundComponents = true;
    }
    else
    //Get next feature
    {
        CComPtr<IFeature> swFeatureNext;
        hres = swFeature->IGetNextFeature(&swFeatureNext);
        swFeature.Release();
        swFeature = swFeatureNext;
    }
}while ( !bFoundComponents);
}

void SuppressSelectedComponent(IComponent2* swComponent, ISldWorks* swApp)
//Suppress selected component if it is resolved
{
    CComBSTR messageSuppressed (L"Component is already suppressed.");

    long IComponentSuppressState;
    long ISuppressMessageResult;
    long ISuppressError;

    hres = swComponent->GetSuppression(&IComponentSuppressState);
    if (IComponentSuppressState == swComponentSuppressed)
        hres = swApp->SendMsgToUser2(messageSuppressed, swMbInformation, swMbOk, &ISuppressMessageResult);
    else
        hres = swComponent->SetSuppression2(swComponentSuppressed, &ISuppressError);
}

void ResolveSelectedComponent(IComponent2* swComponent, ISldWorks* swApp)
//Resolve selected component if it is suppressed
{
    CComBSTR messageResolved (L"Component is already resolved.");

    long IComponentResolveState;
    long IResolveMessageResult;
    long IResolveError;

    hres = swComponent->GetSuppression(&IComponentResolveState);
    if (IComponentResolveState == swComponentFullyResolved)
        hres = swApp->SendMsgToUser2(messageResolved, swMbInformation, swMbOk, &IResolveMessageResult);
    else
        hres = swComponent->SetSuppression2(swComponentFullyResolved, &IResolveError);
}

void ExplodeAssembly(IModelDoc2* swModel, IAssemblyDoc** swAssemblyDoc)
//Explode assembly
{
    CComPtr<IModelView> swExplodeModelView;

```

```

double dZoomFactorOut = 0.3;

IAssemblyDoc* swThisAssemblyDoc;
hres = swModel->get_IActiveView(&swExplodeModelView);
hres = swExplodeModelView->ZoomByFactor(dZoomFactorOut);
hres = swModel->QueryInterface(__uuidof(IAssemblyDoc), reinterpret_cast<void**>(&swThisAssemblyDoc));
hres = swThisAssemblyDoc->AutoExplode(&retVal);
*swAssemblyDoc = swThisAssemblyDoc;
}

void CollapseAssembly(IModelDoc2* swModel, IAssemblyDoc* swAssemblyDoc)
//Collapse assembly
{
    hres = swAssemblyDoc->AutoExplode(&retVal);
    hres = swAssemblyDoc->ViewCollapseAssembly();
    hres = swModel->ViewZoomtofit2();
}

void CreateDrawingOfAssembly(ISldWorks* swApp, IModelDoc2* swModel)
//Create a drawing of the assembly
{
    CComPtr<IModelDoc2> swNewModel;
    CComPtr<IModelDocExtension> swModelDocExt;
    CComPtr<IDrawingDoc> swDrawingDoc;

    CComBSTR sDrawingDocTemplate (L"c:\\Program Files\\SOLIDWORKS\\data\\templates\\Drawing.drwdot");
    CComBSTR sActiveAssembly (L"c:\\test\\universaljoint\\ujoint.sldasm");
    CComBSTR sDrawingDocName (L"ujoint - Sheet1");

    double dWidth = 0.2794;
    double dHeight = 0.4318;
    long lPaperSize = 2;
    long lWindowState = 2;
    long lErrorActivatingDoc;
    hres = swApp->INewDocument2(sDrawingDocTemplate, lPaperSize, dWidth, dHeight, &swNewModel);
    swDrawingDoc = swNewModel;
    hres = swApp->ArrangeWindows(lWindowState);
    hres = swModel->Release();
    hres = swApp->IActivateDoc3(sActiveAssembly, false, &lErrorActivatingDoc, &swModel);
    hres = swModel->get_Extension(&swModelDocExt);
    hres = swModelDocExt->SelectByID2(sActiveAssembly, L"COMPONENT", 0.0, 0.0, 0.0, VARIANT_FALSE, 0, NULL, swSelectOptionDefault, &retVal);
    swNewModel.Release();
    hres = swApp->IActivateDoc3(sDrawingDocName, false, &lErrorActivatingDoc, &swNewModel);
    hres = swDrawingDoc->Create3rdAngleViews(sActiveAssembly, &retVal);
}

void CloseDocuments(ISldWorks* swApp)
//Close assembly and drawing documents
{
    swApp->CloseAllDocuments(true, &retVal);
}

```

stdafx.h: header file

```

/////////////////////////////////////////////////////////////////
//
//stdafx.h: include file for standard system include files,
// or project-specific include files that are used frequently, but
// are changed infrequently
//
/////////////////////////////////////////////////////////////////

#pragma once

#define WIN32_LEAN_AND_MEAN // Exclude rarely-used stuff from Windows headers
#include <stdio.h>
#include <tchar.h>
#define _ATL_CSTRING_EXPLICIT_CONSTRUCTORS // some CString constructors will be explicit

#include <windows.h>
#include <atlbase.h> //Microsoft's ATL helper classes
#include <iostream>

```

```
using namespace std; //Use the standard C++ libraries for text output.
```

```
#import "sldworks.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids //the SOLIDWORKS type library
```

```
#import "swconst.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids //the SOLIDWORKS constant type library
```

```
// TODO: reference additional headers your program requires here
```

Access Data Example (VBA)

[View Code](#)

All features have a feature data object that contains specific information about each instance of the feature. For example, an extrude feature contains an IExtrudeFeatureData2 object that contains the parameters for the extrude's depth. The IExtrudeFeatureData2::AccessSelections method is called to allow a program to modify a feature data object. After making the changes, call IFeature::ModifyDefinition to implement the changes to the feature and release control of the feature data object. A call to IExtrudeFeatureData2::ReleaseSelectionAccess also releases control of the feature data object.

This example shows how to access and modify the data specific to a certain feature in a SOLIDWORKS part or assembly. To use this example, open a part with a base extrusion and select the base extrusion.

1. Attach the program to SOLIDWORKS, get the active model, and access the ISelectionManager.

```
Set swApp = CreateObject("sldWorks.application")  
Set Model = swApp.ActiveDoc  
Set SelMgr = Model.SelectionManager
```

2. Obtain the selected feature, and extract the feature data Object from the feature.

```
Set CurFeature = SelMgr.GetSelectedObject5(1)  
Set FeatData = CurFeature.GetDefinition
```

3. Get the permissions to access the parameters of the feature.

```
isGood = FeatData.AccessSelections(Model, Component)
```

4. Modify the feature data, in this case double the depth of the extrude.

```
Depth = FeatData.GetDepth(True)  
FeatData.SetDepth True, Depth * 2
```

5. Implement the changes in the model.

```
isGood = CurFeature.ModifyDefinition(FeatData, Model, Component)
```

6. If the Feature::ModifyDefinition call fails for any reason, release the access permissions for the feature data.

```
FeatData.ReleaseSelectionAccess
```

Access Selections Example (VBA) - Code

This example doubles the length of the base extrude.

```
Dim swApp As SldWorks.SldWorks
Dim Model As ModelDoc2
Dim Component As Component2
Dim CurFeature As Feature
Dim isGood As Boolean
' Will become an ExtrudeFeatureData Object
Dim FeatData As Object
Dim Depth As Double
Dim SelMgr As SelectionMgr

Sub main()
    Set swApp = CreateObject("sldWorks.application")
    Set Model = swApp.ActiveDoc
    ' Make sure that the active document is a part
    If Model.GetType <> swDocPART And Model.GetType <> swDocASSEMBLY Then
        Msg = "Only allowed on parts or assemblies" ' Define message
        Style = vbOKOnly ' OK button only
        Title = "Error" ' Define title
        Call MsgBox(Msg, Style, Title) ' Display error message
        Exit Sub ' Exit this program
    End If

    ' Get the SelectionManager
    Set SelMgr = Model.SelectionManager

    ' Get the selected object (first in the group if there are more than one)
    ' At this point CurFeature is just a Feature object
    Set CurFeature = SelMgr.GetSelectedObject6(1, 0)
    If CurFeature Is Nothing Then
        ' Tell the user that nothing is selected
        swApp.SendMsgToUser2 "Please select the Base-Extrude.", swMbWarning, swMbOk
        Exit Sub
    End If

    ' Get the component of the selected feature
    ' Needed for AccessSelections()
    Set Component = SelMgr.GetSelectedObjectsComponent3(1, 0)

    ' Check the feature's type name
    ' Make sure it is an extrusion
    If Not CurFeature.GetType = "Extrusion" Then
        swApp.SendMsgToUser2 "Please select the Base-Extrude.", swMbWarning, swMbOk
        Exit Sub
    End If

    ' Get the extrusion's FeatureData object
    Set FeatData = CurFeature.GetDefinition

    ' Get the access selections for the FeatureData object
    ' The component is NULL when accessing the selections of a standalone part
    ' If you are calling AccessSelections from within an assembly, then model
    ' would refer to the top-level document in the assembly and component
    ' would refer to the actual part
    isGood = FeatData.AccessSelections(Model, Component)

    ' Inform the user of an error
    If Not isGood Then
        swApp.SendMsgToUser2 "Unable to obtain access selections.", swMbWarning, swMbOk
        Exit Sub
    End If

    ' Change the depth of this extrusion to double its previous depth
    Depth = FeatData.GetDepth(True)
    FeatData.SetDepth True, Depth * 2
```

' Implement the changes to the feature

isGood = CurFeature.**ModifyDefinition**(FeatData, Model, Component)

' If the modify definition failed

If Not isGood Then

swApp.**SendMsgToUser2** "Unable to modify feature.", swMbWarning, swMbOk

' Release the selections

FeatData.**ReleaseSelectionAccess**

End If

End Sub

Access Structure System Corner Treatments Example (C#)

This example shows how to access structure system corner treatments.

```
// =====
// Preconditions:
// 1. Open public_documents\samples\tutorial\weldments\weldment_box.sldprt.
// 2. Open an Immediate window.
// 3. Press F5.
//
// Postconditions:
// 1. Creates Structure System1.
// 2. Configures its weldment profile to have <GB><round-rods><ROUND BAR 23>.
// 3. Creates Corner Management1 containing:
//    - Two member Corner Group1
//    - Simple Corner Group1
//    - Complex Corner Group1
// 4. Press F5 to inspect the corner treatments.
// 5. Inspect the Immediate window.
// 6. Inspect the structure system and its corner management folder in the FeatureManager design tree.
// =====

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Security;
using System.Text;
using System.Threading.Tasks;
using Microsoft.VisualBasic;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace AccessCorners_CSharp
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 modDoc;
        private FeatureManager swFeatMgr;
        private SelectionMgr swSelMgr;
        private ModelDocExtension modDocExt;
        private StructureSystemMemberFeatureData structMemDef;
        private StructureSystemMemberProfile profDef;
        private PrimaryStructuralMemberFeatureData PrimDef;
        private PrimaryMemberPathSegmentFeatureData memPathSegDef;
        private object[] segments = new object[6];
        private object[] tsegments = new object[1];
        private object[] csegments = new object[2];
        private bool stat;
        private StructureSystemMemberFeatureData[] memData = new StructureSystemMemberFeatureData[1];
        private object memDataArray;
        private Feature feat;
        private int i;
        private int k;
        private int l;
        private int m;
        private int x;
```

```

private int Y;
private Feature swFeat;
private ComplexCornerTreatmentFeatureData complexCrnr;
private string strCornerName;
private object[] featArr;
private int Count;
private StructureSystemFolder swStructSysFolder;
private CornerManagementFolder swCornerMgmtFolder;
private object[] grpArr;
private int grpCnt;
private CornerTreatmentGroupFolder crnrGrpFolder;
private object[] crnrArr;
private int crnrCnt;
private Feature crnrFeat;
private CornerTreatmentFeatureData crnrFeat2;
private SimpleCornerTreatmentFeatureData simpleCrnr;
private TwoMemberCornerTreatmentFeatureData twoMbrCrnr;
private CornerMember member;
private object[] varBodyMem;
private object obj;
private int objType;
private CornerMember tmember;
private object[] crnrMbrArr;
private CornerMember crnrMbr;
private Feature feat1;
private bool status;
private CornerMember pmember;
private object[] varPlanarMem;

public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
{
    int ArraySize;
    ArraySize = Objects.GetUpperBound(0);
    Dispatcher[] d = new Dispatcher[ArraySize + 1];
    int ArrayIndex;
    for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
        d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
    return d;
}

public void Main()
{
    modDoc = (ModelDoc2)swApp.ActiveDoc;
    modDocExt = modDoc.Extension;
    swSelMgr = (SelectionMgr)modDoc.SelectionManager;
    swFeatMgr = modDoc.FeatureManager;

    // Create Structure System1
    Debug.Print("Create Structure System1");
    Debug.Print("");

    stat = modDocExt.SelectByID2("Line4@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, true, 0, null/* TODO Change to default(_)
if this is not a reference type */, 0);
    stat = modDocExt.SelectByID2("Line1@Sketch2", "EXTSKETCHSEGMENT", 0, 0, 0, true, 0, null/* TODO Change to default(_) if
this is not a reference type */, 0);
    stat = modDocExt.SelectByID2("Line3@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, true, 0, null/* TODO Change to default(_)
if this is not a reference type */, 0);
    stat = modDocExt.SelectByID2("Line6@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, true, 0, null/* TODO Change to default(_)
if this is not a reference type */, 0);
    stat = modDocExt.SelectByID2("Line3@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, true, 0, null/* TODO Change to default(_) if
this is not a reference type */, 0);
    stat = modDocExt.SelectByID2("Line4@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, true, 0, null/* TODO Change to default(_) if
this is not a reference type */, 0);

    structMemDef = (StructureSystemMemberFeatureData)modDocExt.CreateStructureSystemMemberData(0);

```

```

        Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " +
        structMemDef.StructureSystemMemberType);

        profDef = (StructureSystemMemberProfile)structMemDef.MemberProfile;

        profDef.ProfileStandard = "GB";
        profDef.ProfileType = "round-rods";
        profDef.ProfileSize = "ROUND BAR 23";

        PrimDef = (PrimaryStructuralMemberFeatureData)structMemDef;
        Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " +
        PrimDef.PrimaryMemberType);

        memPathSegDef = (PrimaryMemberPathSegmentFeatureData)PrimDef;

        segments[0] = swSelMgr.GetSelectedObject6(1, 0);
        segments[1] = swSelMgr.GetSelectedObject6(2, 0);
        segments[2] = swSelMgr.GetSelectedObject6(3, 0);
        segments[3] = swSelMgr.GetSelectedObject6(4, 0);
        segments[4] = swSelMgr.GetSelectedObject6(5, 0);
        segments[5] = swSelMgr.GetSelectedObject6(6, 0);

        DispatchWrapper[] dArray;
        dArray = ObjectArrayToDispatchWrapperArray(segments);

        stat = memPathSegDef.SetPathSegments(dArray);
        Debug.Print("Path segments set successfully: " + stat);

        memData[0] = structMemDef;
        memDataArray = memData;

        modDocExt.CreateStructureSystem(memDataArray, null);

        modDoc.ViewZoomtofit();

        System.Diagnostics.Debugger.Break();

        // Inspect corner treatments

        featArr = (object[])swFeatMgr.GetCornerManagementFolders();
        Count = featArr.GetUpperBound(0);

        for (i = 0; i <= Count; i++)
        {
            swFeat = (Feature)featArr[i];
            Debug.Print(swFeat.Name);
            swCornerMgmtFolder = (CornerManagementFolder)swFeat.GetSpecificFeature2();

            swFeat = swCornerMgmtFolder.GetFeature();

            swFeat = (Feature)swCornerMgmtFolder.GetStructureSystemFolder();
            Debug.Print(swFeat.Name);
            swStructSysFolder = (StructureSystemFolder)swFeat.GetSpecificFeature2();
            Debug.Print("Number of profile groups: " + swStructSysFolder.GetProfileGroupFoldersCount());

            grpCnt = swCornerMgmtFolder.GetTreatmentGroupFolderCount();
            Debug.Print("Number of corner treatment groups: " + grpCnt);
            grpArr = (object[])swCornerMgmtFolder.GetTreatmentGroupFolders();

            for (k = 0; k <= grpCnt - 1; k++)
            {
                crnrGrpFolder = (CornerTreatmentGroupFolder)grpArr[k];
                swFeat = crnrGrpFolder.GetFeature();
                Debug.Print("");
            }
        }

```

```

Debug.Print(swFeat.Name);
Debug.Print("Corner group folder type as defined by swCornerType_e: " + crnrGrpFolder.Type);

crnrCnt = crnrGrpFolder.GetCornerTreatmentCount();
Debug.Print("Number of corner treatments: " + crnrCnt);

crnrArr = (object[])crnrGrpFolder.GetCornerTreatments();

for (x = 0; x <= crnrCnt - 1; x++)
{
    crnrFeat = (Feature)crnrArr[x];
    Debug.Print(crnrFeat.Name);

    crnrFeat2 = (CornerTreatmentFeatureData)crnrFeat.GetDefinition();

    Debug.Print("Corner type as defined by swCornerType_e: " + crnrFeat2.CornerType);
    Debug.Print("Ignore corner treatment? " + crnrFeat2.IgnoreCornerTreatment);
    Debug.Print("Allow extension? " + crnrFeat2.AllowExtension);
    Debug.Print("");

    crnrFeat2.AccessSelections(modDoc, null);
    strCornerName = crnrFeat.Name;

    if (crnrFeat2.CornerType == 0)
    {
        Debug.Print("Simple corner treatment");
        simpleCrnr = (SimpleCornerTreatmentFeatureData)crnrFeat2;
        Debug.Print("Corner treatment trim type as defined by swCornerTreatmentTrimType_e: " +
simpleCrnr.CornerTreatmentTrimType);
        Debug.Print("Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " +
simpleCrnr.PlanarTrimOption);
        Debug.Print("Planar trim tool type as defined by swCornerTreatmentPlanarTrimToolType_e: " +
simpleCrnr.PlanarTrimToolType);
        Debug.Print("Weld trim gap: " + simpleCrnr.WeldTrimGap);
    }
    else if (crnrFeat2.CornerType == 1)
    {
        Debug.Print("Two member corner treatment");
        twoMbrCrnr = (TwoMemberCornerTreatmentFeatureData)crnrFeat2;
        Debug.Print("Corner treatment trim type as defined by swCornerTreatmentTrimType_e: " +
twoMbrCrnr.CornerTreatmentTrimType);
        Debug.Print("Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " +
twoMbrCrnr.PlanarTrimOption);
        Debug.Print("Weld trim gap: " + twoMbrCrnr.WeldTrimGap);

        // Change some settings

        twoMbrCrnr.CornerTreatmentTrimType = 1; // Body trim
        twoMbrCrnr.WeldTrimGap = 0.001;

        crnrMbrArr = (object[])twoMbrCrnr.GetCornerGroupMembers();
        Debug.Print("Corner members:");
        for (Y = 0; Y <= 1; Y++)
        {
            crnrMbr = (CornerMember)crnrMbrArr[Y];
            Debug.Print("Trim order: " + crnrMbr.TrimOrder);

            feat1 = (Feature)crnrMbr.GetUnderlyingMemberFeature();
            Debug.Print("Member feature: " + feat1.Name);
        }

        twoMbrCrnr.SwapMembers(); // Make other member the trim tool
    }
    else if (crnrFeat2.CornerType == 2)
    {
        Debug.Print("Complex corner treatment");
    }
}

```

```

        complexCrnr = (ComplexCornerTreatmentFeatureData)crnrFeat2;

        Debug.Print("Planar trim tool type as defined by swCornerTreatmentPlanarTrimToolType_e: " +
complexCrnr.PlanarTrimToolType);
        Debug.Print("Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " +
complexCrnr.PlanarTrimOption);

        varBodyMem = (object[])complexCrnr.GetBodyTrimMembers();

        if (varBodyMem[0] != null)
        {
            for (l = 0; l <= varBodyMem.GetUpperBound(0); l++)
            {
                member = (CornerMember)varBodyMem[l];
                Debug.Print("Body trim member: " + ((Feature)member.GetUnderlyingMemberFeature()).Name);
                Debug.Print("Body trim order: " + member.TrimOrder);
                Debug.Print("Body weld gap: " + member.WeldGap);
            }
        }

        complexCrnr.SetTrimToolMember(varBodyMem[0]);
        obj = complexCrnr.GetTrimToolMember(out objType);

        if (objType == (int)swTrimToolMemberObjectType_e.swTrimToolMemberObjectType_swCornerMember)
            tmember = (CornerMember)obj;
        else if (objType ==
(int)swTrimToolMemberObjectType_e.swTrimToolMemberObjectType_swCornerTreatmentFeature)
        {
            feat = (Feature)obj;
            Debug.Print("Trim tool: " + feat.Name);
        }
        if (tmember != null)
        {
            Debug.Print("Trim tool member: " + ((Feature)(tmember.GetUnderlyingMemberFeature())).Name);
            Debug.Print("Trim tool member trim order: " + tmember.TrimOrder);
            Debug.Print("Trim tool member weld gap: " + tmember.WeldGap);
        }

        varPlanarMem = (object[])complexCrnr.GetPlanarTrimMembers();

        if (varPlanarMem != null)
        {
            for (m = 0; m <= varPlanarMem.GetUpperBound(0); m++)
            {
                pmember = (CornerMember)varPlanarMem[m];
                Debug.Print("Planar trim member: " + ((Feature)pmember.GetUnderlyingMemberFeature()).Name);
                Debug.Print("Planar trim member trim order: " + pmember.TrimOrder);
                Debug.Print("Planar trim member weld gap: " + pmember.WeldGap);
            }
        }
    }

    feat = crnrFeat2.GetFeature();
    status = feat.ModifyDefinition(crnrFeat2, modDoc, null);
    if (status == false)
        crnrFeat2.ReleaseSelectionAccess();
}

}

}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}

```


Access Structure System Corner Treatments Example (VBA)

This example shows how to access structure system corner treatments.

```
'=====
'Preconditions:
'1. Open public_documents\samples\tutorial\weldments\weldment_box.sldprt.
'2. Open an Immediate window.
'3. Press F5.
'
'Postconditions:
'1. Creates Structure System1.
'2. Configures its weldment profile to have <GB><round-rods><ROUND BAR 23>.
'3. Creates Corner Management1 containing:
'   - Two member Corner Group1
'   - Simple Corner Group1
'   - Complex Corner Group1
'4. Press F5 to inspect the corner treatments.
'5. Inspect the Immediate window.
'6. Inspect the structure system and its corner management folder in the FeatureManager design tree.
'=====
```

```
Dim swApp As SldWorks.SldWorks
Dim modDoc As SldWorks.ModelDoc2
Dim swFeatMgr As SldWorks.FeatureManager
Dim swSelMgr As SldWorks.SelectionMgr
Dim modDocExt As SldWorks.ModelDocExtension
Dim structMemDef As SldWorks.StructureSystemMemberFeatureData
Dim profDef As SldWorks.StructureSystemMemberProfile
Dim PrimDef As SldWorks.PrimaryStructuralMemberFeatureData
Dim memPathSegDef As PrimaryMemberPathSegmentFeatureData
Dim segments(5) As Object
Dim tsegments(0) As Object
Dim csegments(1) As Object
Dim stat As Boolean
Dim memData(0) As SldWorks.StructureSystemMemberFeatureData
Dim memdataArray As Variant
Dim feat As SldWorks.Feature
Dim i As Long
Dim k As Long
Dim l As Long
Dim m As Long
Dim x As Long
Dim Y As Long
Dim swFeat As SldWorks.Feature
Dim complexCrnr As SldWorks.ComplexCornerTreatmentFeatureData
Dim strCornerName As String
Dim featArr As Variant
Dim Count As Long
Dim swStructSysFolder As SldWorks.StructureSystemFolder
Dim swCornerMgmtFolder As SldWorks.CornerManagementFolder
Dim grpArr As Variant
Dim grpCnt As Long
Dim crnrGrpFolder As SldWorks.CornerTreatmentGroupFolder
Dim crnrArr As Variant
Dim crnrCnt As Long
Dim crnrFeat As SldWorks.Feature
Dim crnrFeat2 As SldWorks.CornerTreatmentFeatureData
Dim simpleCrnr As SldWorks.SimpleCornerTreatmentFeatureData
Dim twoMbrCrnr As SldWorks.TwoMemberCornerTreatmentFeatureData
Dim member As SldWorks.CornerMember
Dim varBodyMem As Variant
```

```

Dim obj As Object
Dim objType As Long
Dim tmember As SldWorks.CornerMember
Dim crnrMbrArr As Variant
Dim crnrMbr As SldWorks.CornerMember
Dim feat1 As SldWorks.Feature
Dim status As Boolean
Dim pmember As SldWorks.CornerMember
Dim varPlanarMem As Variant

Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set modDoc = swApp.ActiveDoc
    Set modDocExt = modDoc.Extension
    Set swSelMgr = modDoc.SelectionManager
    Set swFeatMgr = modDoc.FeatureManager

    'Create Structure System1
    Debug.Print "Create Structure System1"
    Debug.Print ""

    stat = modDocExt.SelectByID2("Line4@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID2("Line1@Sketch2", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID2("Line3@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID2("Line6@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID2("Line3@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID2("Line4@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)

    Set structMemDef = modDocExt.CreateStructureSystemMemberData(0)

    Debug.Print "Type of structure system member as defined by swStructureSystemMemberType_e: " &
    structMemDef.StructureSystemMemberType

    Set profDef = structMemDef.MemberProfile

    profDef.ProfileStandard = "GB"
    profDef.ProfileType = "round-rods"
    profDef.ProfileSize = "ROUND BAR 23"

    Set PrimDef = structMemDef
    Debug.Print "Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " &
    PrimDef.PrimaryMemberType

    Set memPathSegDef = PrimDef

    Set segments(0) = swSelMgr.GetSelectedObject6(1, 0)
    Set segments(1) = swSelMgr.GetSelectedObject6(2, 0)
    Set segments(2) = swSelMgr.GetSelectedObject6(3, 0)
    Set segments(3) = swSelMgr.GetSelectedObject6(4, 0)
    Set segments(4) = swSelMgr.GetSelectedObject6(5, 0)
    Set segments(5) = swSelMgr.GetSelectedObject6(6, 0)

    stat = memPathSegDef.SetPathSegments(segments)
    Debug.Print "Path segments set successfully: " & stat

    Set memData(0) = structMemDef
    memDataArray = memData

    modDocExt.CreateStructureSystem memDataArray, Nothing

    modDoc.ViewZoomtofit

    Stop

```

```

'Inspect corner treatments

featArr = swFeatMgr.GetCornerManagementFolders()
Count = UBound(featArr)

For i = 0 To Count
    Set swFeat = featArr(i)
    Debug.Print swFeat.Name
    Set swCornerMgmtFolder = swFeat.GetSpecificFeature2

    Set swFeat = swCornerMgmtFolder.GetFeature

    Set swFeat = swCornerMgmtFolder.GetStructureSystemFolder
    Debug.Print swFeat.Name
    Set swStructSysFolder = swFeat.GetSpecificFeature2
    Debug.Print "Number of profile groups: " & swStructSysFolder.GetProfileGroupFoldersCount

    grpCnt = swCornerMgmtFolder.GetTreatmentGroupFolderCount
    Debug.Print "Number of corner treatment groups: " & grpCnt
    grpArr = swCornerMgmtFolder.GetTreatmentGroupFolders()

    For k = 0 To grpCnt - 1
        Set crnrGrpFolder = grpArr(k)
        Set swFeat = crnrGrpFolder.GetFeature
        Debug.Print ""
        Debug.Print swFeat.Name
        Debug.Print "Corner group folder type as defined by swCornerType_e: " & crnrGrpFolder.Type

        crnrCnt = crnrGrpFolder.GetCornerTreatmentCount
        Debug.Print "Number of corner treatments: " & crnrCnt

        crnrArr = crnrGrpFolder.GetCornerTreatments

        For x = 0 To crnrCnt - 1
            Set crnrFeat = crnrArr(x)
            Debug.Print crnrFeat.Name

            Set crnrFeat2 = crnrFeat.GetDefinition

            Debug.Print "Corner type as defined by swCornerType_e: " & crnrFeat2.CornerType
            Debug.Print "Ignore corner treatment? " & crnrFeat2.IgnoreCornerTreatment
            Debug.Print "Allow extension? " & crnrFeat2.AllowExtension
            Debug.Print ""

            crnrFeat2.AccessSelections modDoc, Nothing
            strCornerName = crnrFeat.Name

            If crnrFeat2.CornerType = 0 Then 'Simple corner treatment
                Debug.Print "Simple corner treatment"
                Set simpleCrnr = crnrFeat2
                Debug.Print "Corner treatment trim type as defined by swCornerTreatmentTrimType_e: " &
simpleCrnr.CornerTreatmentTrimType
                Debug.Print "Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " &
simpleCrnr.PlanarTrimOption
                Debug.Print "Planar trim tool type as defined by swCornerTreatmentPlanarTrimToolType_e: " &
simpleCrnr.PlanarTrimToolType
                Debug.Print "Weld trim gap: " & simpleCrnr.WeldTrimGap

            ElseIf crnrFeat2.CornerType = 1 Then 'Two member corner treatment
                Debug.Print "Two member corner treatment"
                Set twoMbrCrnr = crnrFeat2
                Debug.Print "Corner treatment trim type as defined by swCornerTreatmentTrimType_e: " &
twoMbrCrnr.CornerTreatmentTrimType
                Debug.Print "Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " &
twoMbrCrnr.PlanarTrimOption

```

```

Debug.Print "Weld trim gap: " & twoMbrCnr.WeldTrimGap

'Change some settings

twoMbrCnr.CornerTreatmentTrimType = 1 'Body trim
twoMbrCnr.WeldTrimGap = 0.001

crrMbrArr = twoMbrCnr.GetCornerGroupMembers
Debug.Print "Corner members:"
For Y = 0 To 1
    Set crrMbr = crrMbrArr(Y)
    Debug.Print "Trim order: " & crrMbr.TrimOrder

    Set feat1 = crrMbr.GetUnderlyingMemberFeature()
    Debug.Print "Member feature: " & feat1.Name

Next Y

twoMbrCnr.SwapMembers 'Make other member the trim tool

ElseIf crrFeat2.CornerType = 2 Then 'Complex corner treatment
    Debug.Print "Complex corner treatment"
    Set complexCnr = crrFeat2

    Debug.Print "Planar trim tool type as defined by swCornerTreatmentPlanarTrimToolType_e: " &
complexCnr.PlanarTrimToolType
    Debug.Print "Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " &
complexCnr.PlanarTrimOption

    varBodyMem = complexCnr.GetBodyTrimMembers

    If Not IsEmpty(varBodyMem) Then
        For l = 0 To UBound(varBodyMem)
            Set member = varBodyMem(l)
            Debug.Print "Body trim member: " & member.GetUnderlyingMemberFeature().Name
            Debug.Print "Body trim order: " & member.TrimOrder
            Debug.Print "Body weld gap: " & member.WeldGap
        Next l
    End If

    complexCnr.SetTrimToolMember (varBodyMem(0))
    Set obj = complexCnr.GetTrimToolMember(objType)

    If objType = swTrimToolMemberObjectType_swCornerMember Then
        Set tmember = obj
    ElseIf objType = swTrimToolMemberObjectType_swCornerTreatmentFeature Then
        Set feat = obj
        Debug.Print "Trim tool: " & feat.Name
    End If

    If Not tmember Is Nothing Then
        Debug.Print "Trim tool member: " & tmember.GetUnderlyingMemberFeature().Name
        Debug.Print "Trim tool member trim order: " & tmember.TrimOrder
        Debug.Print "Trim tool member weld gap: " & tmember.WeldGap
    End If

    varPlanarMem = complexCnr.GetPlanarTrimMembers

    If Not IsEmpty(varPlanarMem) Then
        For m = 0 To UBound(varPlanarMem)
            Set pmember = varPlanarMem(m)
            Debug.Print "Planar trim member: " & pmember.GetUnderlyingMemberFeature().Name
            Debug.Print "Planar trim member trim order: " & pmember.TrimOrder
            Debug.Print "Planar trim member weld gap: " & pmember.WeldGap
        Next m
    End If

```

```

'Below is valid only if PlanarTrimToolType is set to
'swCornerTreatmentPlanarTrimToolType_e.swCornerTreatmentPlanarTrimTool_UserDefined

'faces = complexCrnr.UserDefinedTrimFaces

'If Not IsEmpty(faces) Then
    'Set face = faces(0)
    'Debug.Print "User defined trim face area: " & face.GetArea
'End If

End If

Set feat = crnrFeat2.GetFeature()
status = feat.ModifyDefinition(crnrFeat2, modDoc, Nothing)
If status = False Then
    crnrFeat2.ReleaseSelectionAccess
End If

Next x
Next k
Next i

End Sub

```

Access Structure System Corner Treatments Example (VB.NET)

This example shows how to access structure system corner treatments.

```
'=====
'Preconditions:
'1. Open public_documents\samples\tutorial\weldments\weldment_box.sldprt.
'2. Open an Immediate window.
'3. Press F5.
'
'Postconditions:
'1. Creates Structure System1.
'2. Configures its weldment profile to have <GB><round-rods><ROUND BAR 23>.
'3. Creates Corner Management1 containing:
'   - Two member Corner Group1
'   - Simple Corner Group1
'   - Complex Corner Group1
'4. Press F5 to inspect the corner treatments.
'5. Inspect the Immediate window.
'6. Inspect the structure system and its corner management folder in the FeatureManager design tree.
'=====
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim modDoc As ModelDoc2
    Dim swFeatMgr As FeatureManager
    Dim swSelMgr As SelectionMgr
    Dim modDocExt As ModelDocExtension
    Dim structMemDef As StructureSystemMemberFeatureData
    Dim profDef As StructureSystemMemberProfile
    Dim PrimDef As PrimaryStructuralMemberFeatureData
    Dim memPathSegDef As PrimaryMemberPathSegmentFeatureData
    Dim segments(5) As Object
    Dim tsegments(0) As Object
    Dim csegments(1) As Object
    Dim stat As Boolean
    Dim memData(0) As StructureSystemMemberFeatureData
    Dim memDataArray As Object
    Dim feat As Feature
    Dim i As Integer
    Dim k As Integer
    Dim l As Integer
    Dim m As Integer
    Dim x As Integer
    Dim Y As Integer
    Dim swFeat As Feature
    Dim complexCrnr As ComplexCornerTreatmentFeatureData
    Dim strCornerName As String
    Dim featArr As Object
    Dim Count As Integer
    Dim swStructSysFolder As StructureSystemFolder
    Dim swCornerMgmtFolder As CornerManagementFolder
    Dim grpArr As Object
    Dim grpCnt As Integer
    Dim crnrGrpFolder As CornerTreatmentGroupFolder
    Dim crnrArr As Object
    Dim crnrCnt As Integer
    Dim crnrFeat As Feature
```

```

Dim crnrFeat2 As CornerTreatmentFeatureData
Dim simpleCrnr As SimpleCornerTreatmentFeatureData
Dim twoMbrCrnr As TwoMemberCornerTreatmentFeatureData
Dim member As CornerMember
Dim varBodyMem As Object
Dim obj As Object
Dim objType As Integer
Dim tmember As CornerMember
Dim crnrMbrArr As Object
Dim crnrMbr As CornerMember
Dim feat1 As Feature
Dim status As Boolean
Dim pmember As CornerMember
Dim varPlanarMem As Object

Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
    Dim ArraySize As Integer
    ArraySize = Objects.GetUpperBound(0)
    Dim d(ArraySize) As DispatchWrapper
    Dim ArrayIndex As Integer
    For ArrayIndex = 0 To ArraySize
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))
    Next
    Return d
End Function

Sub main()

    modDoc = swApp.ActiveDoc
    modDocExt = modDoc.Extension
    swSelMgr = modDoc.SelectionManager
    swFeatMgr = modDoc.FeatureManager

    'Create Structure System1
    Debug.Print("Create Structure System1")
    Debug.Print("")

    stat = modDocExt.SelectByID("Line4@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID("Line1@Sketch2", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID("Line3@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID("Line6@3DSketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID("Line3@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID("Line4@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)

    structMemDef = modDocExt.CreateStructureSystemMemberData(0)

    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " &
    structMemDef.StructureSystemMemberType)

    profDef = structMemDef.MemberProfile

    profDef.ProfileStandard = "GB"
    profDef.ProfileType = "round-rods"
    profDef.ProfileSize = "ROUND BAR 23"

    PrimDef = structMemDef
    Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " &
    PrimDef.PrimaryMemberType)

    memPathSegDef = PrimDef

    segments(0) = swSelMgr.GetSelectedObject6(1, 0)
    segments(1) = swSelMgr.GetSelectedObject6(2, 0)
    segments(2) = swSelMgr.GetSelectedObject6(3, 0)

```

```

segments(3) = swSelMgr.GetSelectedObject6(4, 0)
segments(4) = swSelMgr.GetSelectedObject6(5, 0)
segments(5) = swSelMgr.GetSelectedObject6(6, 0)

Dim dArray() As DispatchWrapper
dArray = ObjectArrayToDispatchWrapperArray(segments)

stat = memPathSegDef.SetPathSegments(dArray)
Debug.Print("Path segments set successfully: " & stat)

memData(0) = structMemDef
memDataArray = memData

modDocExt.CreateStructureSystem(memDataArray, Nothing)

modDoc.ViewZoomtofit

Stop

'Inspect corner treatments

featArr = swFeatMgr.GetCornerManagementFolders()
Count = UBound(featArr)

For i = 0 To Count
    swFeat = featArr(i)
    Debug.Print(swFeat.Name)
    swCornerMgmtFolder = swFeat.GetSpecificFeature2

    swFeat = swCornerMgmtFolder.GetFeature

    swFeat = swCornerMgmtFolder.GetStructureSystemFolder
    Debug.Print(swFeat.Name)
    swStructSysFolder = swFeat.GetSpecificFeature2
    Debug.Print("Number of profile groups: " & swStructSysFolder.GetProfileGroupFoldersCount)

    grpCnt = swCornerMgmtFolder.GetTreatmentGroupFolderCount
    Debug.Print("Number of corner treatment groups: " & grpCnt)
    grpArr = swCornerMgmtFolder.GetTreatmentGroupFolders()

    For k = 0 To grpCnt - 1
        crnrGrpFolder = grpArr(k)
        swFeat = crnrGrpFolder.GetFeature
        Debug.Print("")
        Debug.Print(swFeat.Name)
        Debug.Print("Corner group folder type as defined by swCornerType_e: " & crnrGrpFolder.Type)

        crnrCnt = crnrGrpFolder.GetCornerTreatmentCount
        Debug.Print("Number of corner treatments: " & crnrCnt)

        crnrArr = crnrGrpFolder.GetCornerTreatments

        For x = 0 To crnrCnt - 1
            crnrFeat = crnrArr(x)
            Debug.Print(crnrFeat.Name)

            crnrFeat2 = crnrFeat.GetDefinition

            Debug.Print("Corner type as defined by swCornerType_e: " & crnrFeat2.CornerType)
            Debug.Print("Ignore corner treatment? " & crnrFeat2.IgnoreCornerTreatment)
            Debug.Print("Allow extension? " & crnrFeat2.AllowExtension)
            Debug.Print("")

            crnrFeat2.AccessSelections(modDoc, Nothing)

```

```

strCornerName = crnrFeat.Name

If crnrFeat2.CornerType = 0 Then 'Simple corner treatment
    Debug.Print("Simple corner treatment")
    simpleCrnr = crnrFeat2
    Debug.Print("Corner treatment trim type as defined by swCornerTreatmentTrimType_e: " &
simpleCrnr.CornerTreatmentTrimType)
    Debug.Print("Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " &
simpleCrnr.PlanarTrimOption)
    Debug.Print("Planar trim tool type as defined by swCornerTreatmentPlanarTrimToolType_e: " &
simpleCrnr.PlanarTrimToolType)
    Debug.Print("Weld trim gap: " & simpleCrnr.WeldTrimGap)

    ElseIf crnrFeat2.CornerType = 1 Then 'Two member corner treatment
        Debug.Print("Two member corner treatment")
        twoMbrCrnr = crnrFeat2
        Debug.Print("Corner treatment trim type as defined by swCornerTreatmentTrimType_e: " &
twoMbrCrnr.CornerTreatmentTrimType)
        Debug.Print("Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " &
twoMbrCrnr.PlanarTrimOption)
        Debug.Print("Weld trim gap: " & twoMbrCrnr.WeldTrimGap)

        'Change some settings

        twoMbrCrnr.CornerTreatmentTrimType = 1 'Body trim
        twoMbrCrnr.WeldTrimGap = 0.001

        crnrMbrArr = twoMbrCrnr.GetCornerGroupMembers
        Debug.Print("Corner members:")
        For Y = 0 To 1
            crnrMbr = crnrMbrArr(Y)
            Debug.Print("Trim order: " & crnrMbr.TrimOrder)

            feat1 = crnrMbr.GetUnderlyingMemberFeature()
            Debug.Print("Member feature: " & feat1.Name)

        Next Y

        twoMbrCrnr.SwapMembers 'Make other member the trim tool

    ElseIf crnrFeat2.CornerType = 2 Then 'Complex corner treatment
        Debug.Print("Complex corner treatment")
        complexCrnr = crnrFeat2

        Debug.Print("Planar trim tool type as defined by swCornerTreatmentPlanarTrimToolType_e: " &
complexCrnr.PlanarTrimToolType)
        Debug.Print("Planar trim option as defined by swCornerTreatmentPlanarTrimOptions_e: " &
complexCrnr.PlanarTrimOption)

        varBodyMem = complexCrnr.GetBodyTrimMembers

        If varBodyMem(0) IsNot Nothing Then
            For l = 0 To UBound(varBodyMem)
                member = varBodyMem(l)
                Debug.Print("Body trim member: " & member.GetUnderlyingMemberFeature().Name)
                Debug.Print("Body trim order: " & member.TrimOrder)
                Debug.Print("Body weld gap: " & member.WeldGap)
            Next l
        End If

        complexCrnr.SetTrimToolMember(varBodyMem(0))
        obj = complexCrnr.GetTrimToolMember(objType)

        If objType = swTrimToolMemberObjectType_e.swTrimToolMemberObjectType_swCornerMember Then
            tmember = obj
        ElseIf objType = swTrimToolMemberObjectType_e.swTrimToolMemberObjectType_swCornerTreatmentFeature Then
            feat = obj

```

```

        Debug.Print("Trim tool: " & feat.Name)
    End If
    If Not tmember Is Nothing Then
        Debug.Print("Trim tool member: " & tmember.GetUnderlyingMemberFeature().Name)
        Debug.Print("Trim tool member trim order: " & tmember.TrimOrder)
        Debug.Print("Trim tool member weld gap: " & tmember.WeldGap)
    End If

    varPlanarMem = complexCnr.GetPlanarTrimMembers

    If varPlanarMem IsNot Nothing Then
        For m = 0 To UBound(varPlanarMem)
            pmember = varPlanarMem(m)
            Debug.Print("Planar trim member: " & pmember.GetUnderlyingMemberFeature().Name)
            Debug.Print("Planar trim member trim order: " & pmember.TrimOrder)
            Debug.Print("Planar trim member weld gap: " & pmember.WeldGap)
        Next m
    End If

    'Below is valid only if PlanarTrimToolType is set to
    'swCornerTreatmentPlanarTrimToolType_e.swCornerTreatmentPlanarTrimTool_UserDefined

    'faces = complexCnr.UserDefinedTrimFaces

    'If Not IsEmpty(faces) Then
    'Set face = faces(0)
    'Debug.Print "User defined trim face area: " & face.GetArea
    'End If

End If

    feat = crnrFeat2.GetFeature()
    status = feat.ModifyDefinition(crnrFeat2, modDoc, Nothing)
    If status = False Then
        crnrFeat2.ReleaseSelectionAccess
    End If

Next x
Next k
Next i

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Activate Each View on Current Sheet Example (VBA)

This example shows how to activate each drawing view on the current sheet.

```
'-----
' Preconditions:
' 1. Open a drawing with one or more drawing views.
' 2. Click a drawing view to activate it.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Activates each drawing view on the current sheet.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swSheet As SldWorks.Sheet
    Dim swView As SldWorks.View
    Dim swActiveView As SldWorks.View
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSheet = swDraw.GetCurrentSheet
    Set swActiveView = swDraw.ActiveDrawingView
    Set swView = swDraw.GetFirstView

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swSheet.GetName
    Debug.Print " Active view = " & swActiveView.GetName2
    Debug.Print ""

    While Not swView Is Nothing
        Debug.Print " " & swView.GetName2 & " [" & swView.Type & "]"
        ' Returns false if trying to activate the drawing sheet
        bRet = swDraw.ActiveView(swView.GetName2):
        If False = bRet Then
            Debug.Assert swSheet.GetName = swView.GetName2
            bRet = swDraw.ActivateSheet(swView.GetName2)
        End If
        Debug.Assert bRet
        Set swView = swView.GetNextView
    Wend

End Sub
```

Activate Feature Example (VBA)

This example shows how to show the dimensions of the selected feature by activating the feature.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Select the feature for which you want to show  
'    its dimensions.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Displays the selected feature's dimensions.  
' 2. Examine the graphics area and Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swSelMgr              As SldWorks.SelectionMgr  
    Dim swFeat               As SldWorks.Feature  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    swModel.ActivateSelectedFeature  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "  Feature = " & swFeat.Name & " [" & swFeat.GetTypeName & "]"  
  
End Sub
```

Activate OLE Object Example (VBA)

This example shows how to activate an OLE object on the active document.

```
'-----  
' Preconditions:  
' 1. Open a model document that contains  
'    Microsoft OLE object.  
' 2. Select the Microsoft OLE object.  
'  
' Postconditions: Opens the worksheet in  
' Microsoft Excel.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swOLEObj As SldWorks.SwOLEObject  
Dim xlObj As Object  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
  
    ' Get selected Microsoft Excel worksheet  
    Set swOLEObj = swSelMgr.GetSelectedObject6(1, -1)  
  
    ' Open the worksheet in Microsoft Excel  
    Set xlObj = swOLEObj.SetActive(True)  
  
End Sub
```

Activate PropertyManager Page Tab Example (C#)

The following code example demonstrates how SOLIDWORKS add-ins can use IPropertyManagerPageTab.Activate to programmatically select a tab on a SOLIDWORKS PropertyManager page.

```
//-----
// Preconditions:
// 1. Copy Main to the main module.
// 2. Click Project > Add Class > Add and copy Class1 to Class1.cs.
// 3. Click Project > Add Reference, browse to install_dir\api\redist\, select
// SolidWorks.Interop.swpublished.dll > OK.
// 4. Open an assembly that has multiple components.
//
// Postconditions:
// 1. Creates a PropertyManager page called Materials and Dimensions
// with two tabs:
// * Materials
// * Dimensions
// 2. Selects the Materials tab.
// 3. Examine the PropertyManager page.
//-----
//Main
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public SldWorks swApp;
        public ModelDoc2 Part;
        public SelectionMgr SelMgr;
        public Class1 pm;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            SelMgr = (SelectionMgr)Part.SelectionManager;

            //Create a new instance of the PropertyManager class
            pm = new Class1(swApp);
            pm.Show();
        }
    }
}

//Class1
using System;
using System.Collections.Generic;
using System.Text;
using System.Runtime.InteropServices;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace Macro1CSharp.csproj
{
    [ComVisibleAttribute(true)]
    public class Class1 : PropertyManagerPage2Handler6
    {
        PropertyManagerPage2 pm_Page;
        bool ClickedCancel;
        public void Show()
        {
            pm_Page.Show();
        }
    }

    public Class1(SolidWorks.Interop.sldworks.SldWorks swApp)
    {
        //General objects required for the PropertyManager page
        PropertyManagerPageTab pm_Page_Tab = default(PropertyManagerPageTab);
        PropertyManagerPageTab pm_Page_Tab_2 = default(PropertyManagerPageTab);
        PropertyManagerPageGroup pm_Group = default(PropertyManagerPageGroup);
        PropertyManagerPageSelectionbox pm_Selection = default(PropertyManagerPageSelectionbox);

        //Each object in the page needs a unique ID
        const int GroupID = 1;
        const int SelectionID = 3;
        const int Tab1ID = 1;
        const int Tab2ID = 2;
        string PageTitle = null;
        string caption = null;
        string tip = null;
        int options = 0;
        int longerrors = 0;
        short controlType = 0;
        short alignment = 0;

        //Set the variables for the page
        PageTitle = "Materials and Dimensions";
        options = (int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_OkayButton + (int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_CancelButton + (int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_LockedPage + (int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton;
        pm_Page = (PropertyManagerPage2)swApp.CreatePropertyManagerPage(PageTitle, options, this, ref longerrors);

        // Create page tabs
        pm_Page_Tab = (PropertyManagerPageTab)pm_Page.AddTab(Tab1ID, "Materials", "", 0);
        pm_Page_Tab_2 = (PropertyManagerPageTab)pm_Page.AddTab(Tab2ID, "Dimensions", "", 0);
    }
}
```

[Back to top](#)

```

// Activate the first tab
pm_Page_Tab.Activate();

//Begin adding the controls to the page tab

//Create the group box
caption = "Materials";
options = (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded;
pm_Group = (PropertyManagerPageGroup)pm_Page_Tab.AddGroupBox(GroupID, caption, options);

//Create selection box
controlType = (short)swPropertyManagerPageControlType_e.swControlType_Selectionbox;
caption = "";

// No caption for selection boxes
alignment = (short)swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent;
options = (int)swAddControlOptions_e.swControlOptions_Visible + (int)swAddControlOptions_e.swControlOptions_Enabled;
tip = "Select an edge, face, vertex, solid body, or a component";
pm_Selection = (PropertyManagerPageSelectionbox)pm_Group.AddControl(SelectionID, controlType, caption, alignment, options, tip);
long[] filters = new long[7];
filters[0] = (long)swSelectType_e.swSelEDGES;
filters[1] = (long)swSelectType_e.swSelREFEDGES;
filters[2] = (long)swSelectType_e.swSelFACES;
filters[3] = (long)swSelectType_e.swSelVERTICES;
filters[4] = (long)swSelectType_e.swSelSOLIDBODIES;
filters[5] = (long)swSelectType_e.swSelCOMPONENTS;
filters[6] = (long)swSelectType_e.swSelCOMPSDONTTOVERRIDE;

pm_Selection.SingleEntityOnly = false;
pm_Selection.AllowMultipleSelectOfSameEntity = true;
pm_Selection.Height = 50;
pm_Selection.SetSelectionFilters(filters);
}

private void PropertyManagerPage2Handler6_AfterActivation()
{
}

private void PropertyManagerPage2Handler6_AfterClose()
{
}

private void PropertyManagerPage2Handler6_OnClose(int Reason)
{
    if (Reason == (int)swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel)
    {
        //Cancel button was clicked
        ClickedCancel = true;
    }
    else if (Reason == (int)swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay)
    {
        //OK button was clicked
        ClickedCancel = false;
    }

    //Store the density and material name values based
    //on the combo-box selection
}

private bool PropertyManagerPage2Handler6_OnHelp()
{
    return false;
}

private bool PropertyManagerPage2Handler6_OnNextPage()
{
    return false;
}

private bool PropertyManagerPage2Handler6_OnPreview()
{
    return false;
}

private bool PropertyManagerPage2Handler6_OnPreviousPage()
{
    return false;
}

private void PropertyManagerPage2Handler6_OnRedo()
{
}

private void PropertyManagerPage2Handler6_OnUndo()
{
}

private void PropertyManagerPage2Handler6_OnWhatsNew()
{
}

public void AfterActivation()
{
}

public void AfterClose()
{
}

public int OnActiveXControlCreated(int Id, bool Status)
{
    return 0;
}

public void OnButtonPress(int Id)
{
}

public void OnCheckboxCheck(int Id, bool Checked)
{
}

public void OnClose(int Reason)
{
}

public void OnComboboxEditChanged(int Id, string Text)
{
}

```

```

    }

    public void OnComboboxSelectionChanged(int Id, int Item)
    {
    }

    public void OnGroupCheck(int Id, bool Checked)
    {
    }

    public void OnGroupExpand(int Id, bool Expanded)
    {
    }

    public bool OnHelp()
    {
        return false;
    }

    public bool OnKeystroke(int Wparam, int Message, int Lparam, int Id)
    {
        return false;
    }

    public void OnListboxSelectionChanged(int Id, int Item)
    {
    }

    public bool OnNextPage()
    {
        return false;
    }

    public void OnNumberboxChanged(int Id, double Value)
    {
    }

    public void OnOptionCheck(int Id)
    {
    }

    public void OnPopupMenuItem(int Id)
    {
    }

    public void OnPopupMenuItemUpdate(int Id, ref int retval)
    {
    }

    public bool OnPreview()
    {
        return false;
    }

    public bool OnPreviousPage()
    {
        return false;
    }

    public void OnRedo()
    {
    }

    public void OnSelectionboxCalloutCreated(int Id)
    {
    }

    public void OnSelectionboxCalloutDestroyed(int Id)
    {
    }

    public void OnSelectionboxFocusChanged(int Id)
    {
    }

    public void OnSelectionboxListChanged(int Id, int Count)
    {
    }

    public void OnSliderPositionChanged(int Id, double Value)
    {
    }

    public void OnSliderTrackingCompleted(int Id, double Value)
    {
    }

    public bool OnSubmitSelection(int Id, object Selection, int SelType, ref string ItemText)
    {
        return false;
    }

    public bool OnTabClicked(int Id)
    {
        return false;
    }

    public void OnTextboxChanged(int Id, string Text)
    {
    }

    public void OnUndo()
    {
    }

    public void OnWhatsNew()
    {
    }
}
}

```

[Back to top](#)

Activate PropertyManager Page Tab Example (VBA)

The following code example demonstrates how SOLIDWORKS add-ins can use IPropertyManagerPageTab.Activate to programmatically select a tab on a SOLIDWORKS PropertyManager page.

```
'-----
' Preconditions:
' 1. Copy and paste Main into your macro.
' 2. Click Insert > Class Module and copy and paste Class1 into the module.
' 3. Add a reference to SOLIDWORKS <version> exposed type libraries
'    for add-in use.
' 4. Open an assembly that has multiple components.
'
' Postconditions:
' 1. Creates a PropertyManager page called Materials and Dimensions
'    with two tabs:
'    * Materials
'    * Dimensions
' 2. Selects the Materials tab.
' 3. Examine the PropertyManager page.
'-----
'Main
Option Explicit

Public swApp As SldWorks.SldWorks
Public Part As SldWorks.ModelDoc2
Public SelMgr As SldWorks.SelectionMgr
Public pm As Class1
Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set SelMgr = Part.SelectionManager

    'Create a new instance of the PropertyManager class
    'Calls the Class_Initialize procedure of the class
    Set pm = New Class1
    pm.Show

End Sub

'-----
'Class1
Option Explicit

'Required for PropertyManager page controls
Implements PropertyManagerPage2Handler6

'General objects required for the PropertyManager page
Dim pm_Page As PropertyManagerPage2
Dim pm_Page_Tab As PropertyManagerPageTab
Dim pm_Page_Tab_2 As PropertyManagerPageTab
Dim pm_Group As PropertyManagerPageGroup
Dim pm_Selection As PropertyManagerPageSelectionbox
Dim pm_Label As PropertyManagerPageLabel
Dim pm_Combo As PropertyManagerPageCombobox

'Each object in the page needs a unique ID
Const GroupID As Integer = 1
Const LabelID As Integer = 2
Const SelectionID As Integer = 3
Const ComboID As Integer = 4
Const Tab1ID As Integer = 1
Const Tab2ID As Integer = 2

Dim ClickedCancel As Boolean
Dim density As String
Dim material As String

Sub Show()
    pm_Page.Show
End Sub
Private Sub Class_Initialize()
    Dim PageTitle As String
    Dim caption As String
    Dim tip As String
    Dim options As Long
    Dim longerrors As Long
    Dim controlType As Integer
    Dim alignment As Integer

    'Set the variables for the page
    PageTitle = "Materials and Dimensions"
    options = swPropertyManager_OkayButton + swPropertyManager_CancelButton + swPropertyManagerOptions_LockedPage + swPropertyManagerOptions_PushpinButton

    'Create the PropertyManager page
    Set pm_Page = swApp.CreatePropertyManagerPage(PageTitle, options, Me, longerrors)

    'Make sure that the page was created properly
    If longerrors = swPropertyManagerPage_Okay Then
        ' Create page tabs
        Set pm_Page_Tab = pm_Page.AddTab(Tab1ID, "Materials", "", 0)
        Set pm_Page_Tab_2 = pm_Page.AddTab(Tab2ID, "Dimensions", "", 0)

        ' Activate the first tab
        pm_Page_Tab.Activate
    End If
End Sub
```

[Back to top](#)

```

'Begin adding the controls to the page tab

'Create the group box
caption = "Materials"
options = swGroupBoxOptions_Visible + swGroupBoxOptions_Expanded
Set pm_Group = pm_Page_Tab.AddGroupBox(GroupID, caption, options)

'Create selection box
controlType = swControlType_Selectionbox
caption = "" ' No caption for selection boxes
alignment = swControlAlign_Indent
options = swControlOptions_Visible + swControlOptions_Enabled
tip = "Select an edge, face, vertex, solid body, or a component"
Set pm_Selection = pm_Group.AddControl(SelectionID, controlType, caption, alignment, options, tip)
Dim filters(6) As Long
filters(0) = swSelEDGES
filters(1) = swSelREFEDGES
filters(2) = swSelFACES
filters(3) = swSelVERTICES
filters(4) = swSelSOLIDBODIES
filters(5) = swSelCOMPONENTS
filters(6) = swSelCOMPSDONTTOVERRIDE

pm_Selection.SingleEntityOnly = False
pm_Selection.AllowMultipleSelectOfSameEntity = True
pm_Selection.Height = 50
pm_Selection.SetSelectionFilters filters
Else 'If the page is not created
MsgBox "An error while attempting to create the " & "PropertyManager Page", vbCritical
End If

End Sub

Private Sub PropertyManagerPage2Handler6_AfterActivation()

End Sub

Private Sub PropertyManagerPage2Handler6_AfterClose()

Set pm = Nothing

End Sub

Private Function PropertyManagerPage2Handler6_OnActiveXControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long

End Function

Private Sub PropertyManagerPage2Handler6_OnButtonPress(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler6_OnCheckboxCheck(ByVal Id As Long, ByVal Checked As Boolean)

End Sub

Private Sub PropertyManagerPage2Handler6_OnClose(ByVal Reason As Long)

If Reason = swPropertyManagerPageClose_Cancel Then
'Cancel button was clicked
ClickedCancel = True
ElseIf Reason = swPropertyManagerPageClose_Okay Then
'OK button was clicked
ClickedCancel = False
End If

'Store the density and material name values based
'on the combo-box selection
End Sub

Private Sub PropertyManagerPage2Handler6_OnComboboxEditChanged(ByVal Id As Long, ByVal Text As String)

End Sub

Private Sub PropertyManagerPage2Handler6_OnComboboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)

End Sub

Private Sub PropertyManagerPage2Handler6_OnGroupCheck(ByVal Id As Long, ByVal Checked As Boolean)

End Sub

Private Sub PropertyManagerPage2Handler6_OnGroupExpand(ByVal Id As Long, ByVal Expanded As Boolean)

End Sub

Private Function PropertyManagerPage2Handler6_OnHelp() As Boolean

End Function

Private Function PropertyManagerPage2Handler6_OnKeystroke(ByVal Wparam As Long, ByVal Message As Long, ByVal Lparam As Long, ByVal Id As Long) As Boolean

End Function

Private Sub PropertyManagerPage2Handler6_OnListboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)

End Sub

Private Function PropertyManagerPage2Handler6_OnNextPage() As Boolean

End Function

Private Sub PropertyManagerPage2Handler6_OnNumberboxChanged(ByVal Id As Long, ByVal Value As Double)

End Sub

Private Sub PropertyManagerPage2Handler6_OnOptionCheck(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler6_OnPopupMenuItem(ByVal Id As Long)

```

```

End Sub

Private Sub PropertyManagerPage2Handler6_OnPopupMenuItemSelectedUpdate(ByVal Id As Long, retval As Long)

End Sub

Private Function PropertyManagerPage2Handler6_OnPreview() As Boolean

End Function

Private Function PropertyManagerPage2Handler6_OnPreviousPage() As Boolean

End Function

Private Sub PropertyManagerPage2Handler6_OnRedo()

End Sub

Private Sub PropertyManagerPage2Handler6_OnSelectionboxCalloutCreated(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler6_OnSelectionboxCalloutDestroyed(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler6_OnSelectionboxFocusChanged(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler6_OnSelectionboxListChanged(ByVal Id As Long, ByVal Count As Long)

End Sub

Private Sub PropertyManagerPage2Handler6_OnSliderPositionChanged(ByVal Id As Long, ByVal Value As Double)

End Sub

Private Sub PropertyManagerPage2Handler6_OnSliderTrackingCompleted(ByVal Id As Long, ByVal Value As Double)

End Sub

Private Function PropertyManagerPage2Handler6_OnSubmitSelection(ByVal Id As Long, ByVal Selection As Object, ByVal SelType As Long, ItemText As String) As Boolean
    PropertyManagerPage2Handler6_OnSubmitSelection = True
End Function

Private Function PropertyManagerPage2Handler6_OnTabClicked(ByVal Id As Long) As Boolean

End Function

Private Sub PropertyManagerPage2Handler6_OnTextboxChanged(ByVal Id As Long, ByVal Text As String)

End Sub

Private Sub PropertyManagerPage2Handler6_OnUndo()

End Sub

Private Sub PropertyManagerPage2Handler6_OnWhatsNew()

End Sub

```

[Back to top](#)

Activate PropertyManager Page Tab Example (VB.NET)

The following code example demonstrates how SOLIDWORKS add-ins can use `IPropertyManagerPageTab.Activate` to programmatically select a tab on a SOLIDWORKS PropertyManager page.

```

-----
Preconditions:
1. Copy Main to the main module.
2. Click Project > Add Class > Add and copy Class1 to Class1.vb.
3. Click Project > Add Reference, browse to install_dir\api\redist\, select SolidWorks.Interop.swpublished.dll > OK.
4. Open an assembly that has multiple components.
Postconditions:
1. Creates a PropertyManager page called Materials and Dimensions with two tabs:
   * Materials
   * Dimensions
2. Selects the Materials tab.
3. Examine the PropertyManager page.
-----
'Main
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro
    Public swApp As SldWorks
    Public Part As ModelDoc2
    Public SelMgr As SelectionMgr
    Public pm As Class1

    Public Sub main()
        Part = swApp.ActiveDoc
        SelMgr = CType(Part.SelectionManager, SelectionMgr)
        'Create a new instance of the PropertyManager class
        pm = New Class1(swApp)
        pm.Show()
    End Sub
End Class

Back to top

'Class1
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Runtime.InteropServices

<ComVisibleAttribute(True)> _
Public Class Class1

    Implements PropertyManagerPage2Handler6
    Dim pm_Page As PropertyManagerPage2
    Dim ClickedCancel As Boolean
    Sub Show()
        pm_Page.Show()
    End Sub

    Sub New(ByVal swApp As SolidWorks.Interop.sldworks.SldWorks)
        'General objects required for the PropertyManager page
        Dim pm_Page_Tab As PropertyManagerPageTab
        Dim pm_Page_Tab_2 As PropertyManagerPageTab
        Dim pm_Group As PropertyManagerPageGroup
        Dim pm_Selection As PropertyManagerPageSelectionbox

        'Each object in the page needs a unique ID
        Const GroupID As Integer = 1
        Const SelectionID As Integer = 3
        Const Tab1ID As Integer = 1
        Const Tab2ID As Integer = 2
        Dim PageTitle As String
        Dim caption As String
        Dim tip As String
        Dim options As Integer
        Dim longerrors As Long
        Dim controlType As Integer
        Dim alignment As Integer

        'Set the variables for the page
        PageTitle = "Materials and Dimensions"
        options = swPropertyManagerPageOptions_e.swPropertyManagerOptions_OkayButton + swPropertyManagerPageOptions_e.swPropertyManagerOptions_CancelButton + swPropertyManagerPageOptions_e.swPropertyManagerOptions_LockedPage + swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton
        n + swPropertyManagerPageOptions_e.swPropertyManagerOptions_LockedPage + swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton
        pm_Page = swApp.CreatePropertyManagerPage(PageTitle, options, Me, longerrors)
        ' Create page tabs
        pm_Page_Tab = CType(pm_Page.AddTab(Tab1ID, "Materials", "", 0), PropertyManagerPageTab)
        pm_Page_Tab_2 = CType(pm_Page.AddTab(Tab2ID, "Dimensions", "", 0), PropertyManagerPageTab)
        ' Activate the first tab
        pm_Page_Tab.Activate()
        'Begin adding the controls to the page tab
        'Create the group box
        caption = "Materials"
        options = swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded
        pm_Group = CType(pm_Page_Tab.AddGroupBox(GroupID, caption, options), PropertyManagerPageGroup)

        'Create selection box
        controlType = swPropertyManagerPageControlType_e.swControlType_Selectionbox

        caption = "" ' No caption for selection boxes
    End Sub
End Class

```

```

alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent
options = swAddControlOptions_e.swControlOptions_Visible + swAddControlOptions_e.swControlOptions_Enabled

tip = "Select an edge, face, vertex, solid body, or a component"
pm_Selection = CType(pm_Group.AddControl(SelectionID, controlType, caption, alignment, options, tip), PropertyManagerPageSelectionbox)
Dim filters(6) As Long
filters(0) = swSelectType_e.swSelEdges
filters(1) = swSelectType_e.swSelREFEDGES
filters(2) = swSelectType_e.swSelFACES
filters(3) = swSelectType_e.swSelVERTICES
filters(4) = swSelectType_e.swSelSOLIDBODIES
filters(5) = swSelectType_e.swSelCOMPONENTS
filters(6) = swSelectType_e.swSelCOMPSDONTTOVERRIDE
pm_Selection.SingleEntityOnly = False
pm_Selection.AllowMultipleSelectOfSameEntity = True
pm_Selection.Height = 50
pm_Selection.SetSelectionFilters(filters)

End Sub

Private Sub PropertyManagerPage2Handler6_AfterActivation()
End Sub

Private Sub PropertyManagerPage2Handler6_AfterClose()
End Sub

Private Sub PropertyManagerPage2Handler6_OnClose(ByVal Reason As Integer)
If Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel Then
    'Cancel button was clicked
    ClickedCancel = True
ElseIf Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay Then
    'OK button was clicked
    ClickedCancel = False
End If
'Store the density and material name values based
'on the combo-box selection
End Sub

Private Function PropertyManagerPage2Handler6_OnHelp() As Boolean
End Function

Private Function PropertyManagerPage2Handler6_OnNextPage() As Boolean
End Function

Private Function PropertyManagerPage2Handler6_OnPreview() As Boolean
End Function

Private Function PropertyManagerPage2Handler6_OnPreviousPage() As Boolean
End Function

Private Sub PropertyManagerPage2Handler6_OnRedo()
End Sub

Private Sub PropertyManagerPage2Handler6_OnUndo()
End Sub

Private Sub PropertyManagerPage2Handler6_OnWhatsNew()
End Sub

Public Sub AfterActivation() Implements IPropertyManagerPage2Handler6.AfterActivation
End Sub

Public Sub AfterClose() Implements IPropertyManagerPage2Handler6.AfterClose
End Sub

Public Function OnActiveXControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements IPropertyManagerPage2Handler6.OnActiveXCon
trolCreated
End Function

Public Sub OnButtonPress(ByVal Id As Integer) Implements IPropertyManagerPage2Handler6.OnButtonPress
End Sub

Public Sub OnCheckboxCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements IPropertyManagerPage2Handler6.OnCheckboxCheck
End Sub

Public Sub OnClose(ByVal Reason As Integer) Implements IPropertyManagerPage2Handler6.OnClose
End Sub

Public Sub OnComboboxEditChanged(ByVal Id As Integer, ByVal Text As String) Implements IPropertyManagerPage2Handler6.OnComboboxEditChanged
End Sub

Public Sub OnComboboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements IPropertyManagerPage2Handler6.OnComboboxSelectionChanged
End Sub

Public Sub OnGroupCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements IPropertyManagerPage2Handler6.OnGroupCheck
End Sub

Public Sub OnGroupExpand(ByVal Id As Integer, ByVal Expanded As Boolean) Implements IPropertyManagerPage2Handler6.OnGroupExpand
End Sub

Public Function OnHelp() As Boolean Implements IPropertyManagerPage2Handler6.OnHelp
End Function

Public Function OnKeystroke(ByVal Wparam As Integer, ByVal Message As Integer, ByVal Lparam As Integer, ByVal Id As Integer) As Boolean Implements IPR
opertyManagerPage2Handler6.OnKeystroke
End Function

Public Sub OnListboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements IPropertyManagerPage2Handler6.OnListboxSelectionChanged
End Sub

Public Function OnNextPage() As Boolean Implements IPropertyManagerPage2Handler6.OnNextPage
End Function

Public Sub OnNumberboxChanged(ByVal Id As Integer, ByVal Value As Double) Implements IPropertyManagerPage2Handler6.OnNumberboxChanged
End Sub

Public Sub OnOptionCheck(ByVal Id As Integer) Implements IPropertyManagerPage2Handler6.OnOptionCheck
End Sub

```

```

Public Sub OnPopupMenuItem(ByVal Id As Integer) Implements IPropertyManagerPage2Handler6.OnPopupMenuItem
End Sub

Public Sub OnPopupMenuItemUpdate(ByVal Id As Integer, ByRef retval As Integer) Implements IPropertyManagerPage2Handler6.OnPopupMenuItemUpdate
End Sub

Public Function OnPreview() As Boolean Implements IPropertyManagerPage2Handler6.OnPreview
End Function

Public Function OnPreviousPage() As Boolean Implements IPropertyManagerPage2Handler6.OnPreviousPage
End Function

Public Sub OnRedo() Implements IPropertyManagerPage2Handler6.OnRedo
End Sub

Public Sub OnSelectionboxCalloutCreated(ByVal Id As Integer) Implements IPropertyManagerPage2Handler6.OnSelectionboxCalloutCreated
End Sub

Public Sub OnSelectionboxCalloutDestroyed(ByVal Id As Integer) Implements IPropertyManagerPage2Handler6.OnSelectionboxCalloutDestroyed
End Sub

Public Sub OnSelectionboxFocusChanged(ByVal Id As Integer) Implements IPropertyManagerPage2Handler6.OnSelectionboxFocusChanged
End Sub

Public Sub OnSelectionboxListChanged(ByVal Id As Integer, ByVal Count As Integer) Implements IPropertyManagerPage2Handler6.OnSelectionboxListChanged
End Sub

Public Sub OnSliderPositionChanged(ByVal Id As Integer, ByVal Value As Double) Implements IPropertyManagerPage2Handler6.OnSliderPositionChanged
End Sub

Public Sub OnSliderTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements IPropertyManagerPage2Handler6.OnSliderTrackingCompleted
End Sub

Public Function OnSubmitSelection(ByVal Id As Integer, ByVal Selection As Object, ByVal SelType As Integer, ByRef ItemText As String) As Boolean Implements IPropertyManagerPage2Handler6.OnSubmitSelection
End Function

Public Function OnTabClicked(ByVal Id As Integer) As Boolean Implements IPropertyManagerPage2Handler6.OnTabClicked
End Function

Public Sub OnTextboxChanged(ByVal Id As Integer, ByVal Text As String) Implements IPropertyManagerPage2Handler6.OnTextboxChanged
End Sub

Public Sub OnUndo() Implements IPropertyManagerPage2Handler6.OnUndo
End Sub

Public Sub OnWhatsNew() Implements IPropertyManagerPage2Handler6.OnWhatsNew
End Sub

End Class

```

[Back to top](#)

Activate Smart Feature in an Assembly Example (C#)

This example shows how to activate a Smart Feature in an assembly.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\testSmartAssembly.sldasm.
//
// Postconditions:
// 1. Activates Smart Feature holecube-1 and creates two extruded cuts
// for the selected reference entity.
// 2. Displays Smart Feature holecube-1 in the FeatureManager design tree.
// 3. Creates Smart-Feature1 in component testpart100.
// 4. Expand and examine testpart100 and holecube-1 in the FeatureManager
// design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
```

```
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace ActivateSmartFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        Component2 comp;
        object comps;
        Object[] compsArray;
        SelectionMgr selMgr;
        object features;
        object featuresSelected;
        Boolean[] featuresSelectedArray;
        object components;
        object componentsSelected;
        Boolean[] componentsSelectedArray;
        object references;
        Object[] referenceArray;
        Face2 face;
        AssemblyDoc swAss;
        ModelDoc2 doc;
        bool boolstatus;
        int i;
        int j;

        public void Main()
        {
            swAss = (AssemblyDoc)swApp.ActiveDoc;

            comps = swAss.GetComponents(true);
            compsArray = (Object[])comps;

            doc = (ModelDoc2)swAss;
```

```

selMgr = (SelectionMgr)doc.SelectionManager;

for (i = 0; i <= compsArray.GetUpperBound(0); i++)
{
    comp = (Component2)compsArray[i];

    if (comp.IsSmartComponent())
    {
        if (comp.GetSmartComponentData(out features, out featuresSelected, out components, out componentsSelected, out references))
        {
            if ((featuresSelected == null)

== false)
            {
                featuresSelectedArray = (Boolean[])featuresSelected;
                for (j = 0; j <= featuresSelectedArray.GetUpperBound(0); j++)
                {
                    if (featuresSelectedArray[j] == false)
                    {
                        featuresSelectedArray[j] = true;
                    }
                }
            }

            if ((componentsSelected == null)

== false)
            {
                componentsSelectedArray = (Boolean[])componentsSelected;
                for (j = 0; j <= componentsSelectedArray.GetUpperBound(0); j++)
                {
                    if (componentsSelectedArray[j] == false)
                    {
                        componentsSelectedArray[j - 1] = true;
                    }
                }
            }

            boolstatus = doc.Extension.SelectByID("", "FACE",
-0.1054255613309, -0.008376708432593, 0.03086069829328, false, 0, null, 0);

            face = (Face2)selMgr.GetSelectedObject6(1,
-1);

            referenceArray = (Object[])references;
            referenceArray[0] = face;

            Dispatcher[] arrRefsIn = new Dispatcher[1];
            arrRefsIn[0] = new Dispatcher(referenceArray[0]);

            boolstatus = comp.SetSmartComponentData(featuresSelected, componentsSelected, (arrRefsIn));
        }

        Debug.Print(comp.Name);
    }
}

```

```
}
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Activate Smart Feature in an Assembly Example (VBA)

This example shows how to activate a Smart Feature in an assembly.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\testSmartAssembly.sldasm.
'
' Postconditions:
' 1. Activates Smart Feature holecube-1 and creates two extruded cuts
'   for the selected reference entity.
' 2. Displays Smart Feature holecube-1 in the FeatureManager design tree.
' 3. Creates Smart-Feature1 in component testpart100.
' 4. Expand and examine testpart100 and holecube-1 in the FeatureManager
'   design tree.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
```

```
Dim swApp As SldWorks.SldWorks
Dim feat As SldWorks.Feature
Dim comp As SldWorks.Component2
Dim comps As Variant
Dim selMgr As SelectionMgr
Dim features As Variant
Dim config As SldWorks.Configuration
Dim featuresSelected As Variant
Dim components As Variant
Dim componentsSelected As Variant
Dim references As Variant
Dim face As SldWorks.Face2
Dim swAss As SldWorks.AssemblyDoc
Dim doc As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim i As Long
Dim j As Long

Option Explicit

Sub main()
    Set swApp = Application.SldWorks
    Set swAss = swApp.ActiveDoc

    comps = swAss.GetComponents(True)

    Set doc = swAss
    Set selMgr = doc.SelectionManager

    For i = 0 To UBound(comps)

        Set comp = comps(i)
        If comp.IsSmartComponent Then

            If comp.GetSmartComponentData(features, featuresSelected, components, componentsSelected, references) Then

                If IsEmpty(featuresSelected) = False Then
                    For j = 0 To UBound(featuresSelected)
                        If featuresSelected(j) = False Then
                            featuresSelected(j) = True
                        End If
                    Next
                End If

                If IsEmpty(componentsSelected) = False Then
                    For j = 0 To UBound(componentsSelected)
                        If featuresSelected(j) = False Then
                            componentsSelected(j - 1) = True
                        End If
                    Next
                End If

                boolstatus = swAss.Extension.SelectByID2("", "FACE", -0.1054255613309, -0.008376708432593, 0.03086069829328, False,
0, Nothing, 0)
                Set face = selMgr.GetSelectedObject(1)
                Set references(0) = face

                boolstatus = comp.SetSmartComponentData(featuresSelected, componentsSelected, references)

            End If

        End If

    Next i
End Sub
```

```
Debug.Print comp.Name
```

```
End If
```

```
Next
```

```
End Sub
```

Activate Smart Feature in an Assembly Example (VB.NET)

This example shows how to activate a Smart Feature in an assembly.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\testSmartAssembly.sldasm.
'
' Postconditions:
' 1. Activates Smart Feature holecube-1 and creates two extruded cuts
'    for the selected reference entity.
' 2. Displays Smart Feature holecube-1 in the FeatureManager design tree.
' 3. Creates Smart-Feature1 in component testpart100.
' 4. Expand and examine testpart100 and holecube-1 in the FeatureManager
'    design tree.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```

Partial Class SolidWorksMacro

```

```

    Dim feat As Feature
    Dim comp As Component2
    Dim comps As Object
    Dim selMgr As SelectionMgr
    Dim features As Object
    Dim config As Configuration
    Dim featuresSelected As Object
    Dim components As Object
    Dim componentsSelected As Object
    Dim references As Object
    Dim face As Face2
    Dim swAss As AssemblyDoc
    Dim doc As ModelDoc2
    Dim boolstatus As Boolean
    Dim i As Long
    Dim j As Long

```

```

    Sub main()

```

```

        swAss = swApp.ActiveDoc

```

```

        comps = swAss.GetComponents(True)

```

```

        doc = swAss

```

```

        selMgr = doc.SelectionManager

```

```

        For i = 0 To UBound(comps)

```

```

            comp = comps(i)

```

```

            If comp.IsSmartComponent Then

```

```

                If comp.GetSmartComponentData(features, featuresSelected, components, componentsSelected, references) Then

```

```

                    If IsNothing(featuresSelected) = False Then

```

```

                        For j = 0 To UBound(featuresSelected)

```

```

                            If featuresSelected(j) = False Then

```

```

                                featuresSelected(j) = True

```

```

                            End If

```

```

                        Next

```

```

                    End If

```

```

                    If IsNothing(componentsSelected) = False Then

```

```

                        For j = 0 To UBound(componentsSelected)

```

```

                            If featuresSelected(j) = False Then

```

```

                                componentsSelected(j - 1) = True

```

```

                            End If

```

```

                        Next

```

```

                    End If

```

```

                    boolstatus = doc.Extension.SelectByID2("", "FACE", -0.1054255613309, -0.008376708432593, 0.03086069829328,
False, 0, Nothing, 0)

```

```

                    face = selMgr.GetSelectedObject6(1, -1)

```

```

                    references(0) = face

```

```

                    Dim arrRefsIn(0) As Dispatcher

```

```

                    arrRefsIn(0) = New Dispatcher(references(0))

```

```

                    boolstatus = comp.SetSmartComponentData(featuresSelected, componentsSelected, (arrRefsIn))

```

```

                End If

```

```

            Debug.Print(comp.Name)

```

End If

Next

End Sub

Public swApp As SldWorks

End Class

Activate SOLIDWORKS CommandManager Tab Example (C#)

This example shows how to activate a SOLIDWORKS CommandManager tab.

```
//-----
// Preconditions: Open a part.
//
// Postconditions:
// 1. Inspect the Immediate window for the SOLIDWORKS CommandManager tab
//    information.
// 2. Inspect the CommandManager.
//    * Sketch tab is activated.
//    * Mold Tools tab (tab index 5) is visible.
//-----
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace ActivateCommandManagerTab_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModelDoc;
        ModelDocExtension swModelDocExt;
        bool retval;
        object cmdTabs;
        string activeTab;

        int activeTabIndex;

        public void Main()
        {
            swModelDoc = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModelDoc.Extension;

            //Get SOLIDWORKS CommandManager tabs
            cmdTabs = swModelDocExt.GetCommandTabs();
            //Active CommandManager tab is "Features"
            activeTab = swModelDocExt.ActiveCommandTab;
            Debug.Print("Active tab is " + activeTab);
            activeTabIndex = swModelDocExt.ActiveCommandTabIndex;
            Debug.Print("Active tab index is " + activeTabIndex);
            //Tab with index 5 is not visible
            retval = swModelDocExt.get_CommandTabVisible(5);
            Debug.Print("Tab with index 5 is visible? " + retval);

            //Set the active CommandManager tab
            swModelDocExt.ActiveCommandTab = "Sketch";
            activeTabIndex = swModelDocExt.ActiveCommandTabIndex;
            //Make tab with index 5 visible
            swModelDocExt.set_CommandTabVisible(5, true);
        }

        public SldWorks swApp;
    }
}
```

Activate SOLIDWORKS CommandManager Tab Example (VBA)

This example shows how to activate a SOLIDWORKS CommandManager tab.

```
'-----
' Preconditions: Open a part.
'
' Postconditions:
' 1. Inspect the Immediate window for the SOLIDWORKS CommandManager tab
'    information.
' 2. Inspect the CommandManager.
'    * Sketch tab is activated.
'    * Mold Tools tab (tab index 5) is visible.
'-----
```

```
Dim swApp           As SldWorks.SldWorks
Dim swModelDoc       As SldWorks.ModelDoc2
Dim swModelDocExt    As SldWorks.ModelDocExtension
Dim retval          As Boolean
Dim cmdTabs          As Variant
Dim activeTab        As String
Dim activeTabIndex  As Long
```

Option Explicit

Sub main()

```
Set swApp = Application.SldWorks
Set swModelDoc = swApp.ActiveDoc
Set swModelDocExt = swModelDoc.Extension
```

```
'Get SOLIDWORKS CommandManager tabs
cmdTabs = swModelDocExt.GetCommandTabs
```

```
'Active CommandManager tab is "Features"
activeTab = swModelDocExt.ActiveCommandTab
Debug.Print "Active tab is " & activeTab
```

```
activeTabIndex = swModelDocExt.ActiveCommandTabIndex
Debug.Print "Active tab index is " & activeTabIndex
```

```
'Tab with index 5 is not visible
retval = swModelDocExt.CommandTabVisible(5)
Debug.Print "Tab with index 5 is visible? " & retval
```

```
'Set the active CommandManager tab
swModelDocExt.ActiveCommandTab = "Sketch"
activeTabIndex = swModelDocExt.ActiveCommandTabIndex
```

```
'Make tab with index 5 visible
swModelDocExt.CommandTabVisible(5) = True
```

End Sub

Activate SOLIDWORKS CommandManager Tab Example (VB.NET)

This example shows how to activate a SOLIDWORKS CommandManager tab.

```
'-----
' Preconditions: Open a part.
'
' Postconditions:
' 1. Inspect the Immediate window for the SOLIDWORKS CommandManager tab
'    information.
' 2. Inspect the CommandManager.
'    * Sketch tab is activated.
'    * Mold Tools tab (tab index 5) is visible.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModelDoc As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim retval As Boolean
    Dim cmdTabs As Object
    Dim activeTab As String
    Dim activeTabIndex As Integer

    Sub main()

        swModelDoc = swApp.ActiveDoc
        swModelDocExt = swModelDoc.Extension

        'Get SOLIDWORKS CommandManager tabs
        cmdTabs = swModelDocExt.GetCommandTabs
        'Active CommandManager tab is "Features"
        activeTab = swModelDocExt.ActiveCommandTab
        Debug.Print("Active tab is " & activeTab)
        activeTabIndex = swModelDocExt.ActiveCommandTabIndex
        Debug.Print("Active tab index is " & activeTabIndex)
        'Tab with index 5 is not visible
        retval = swModelDocExt.CommandTabVisible(5)
        Debug.Print("Tab with index 5 is visible? " & retval)

        'Set the active CommandManager tab
        swModelDocExt.ActiveCommandTab = "Sketch"
        activeTabIndex = swModelDocExt.ActiveCommandTabIndex
        'Make tab with index 5 visible
        swModelDocExt.CommandTabVisible(5) = True
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Add .NET Controls to SOLIDWORKS Design using an Add-in Example (C#)

This example shows how to add .NET controls to FeatureManager, PropertyManager, Task Pane, and model view using an add-in.

```
//-----
// Preconditions:
// 1. In Microsoft Visual Studio 2012 or later, create a new project using
//    the SwCSharpAddin template.
// 2. Name the project DotNetControlsDemo.
// 3. Select Debug in the Solution Configurations drop-down list.
// 4. Click Project > DotNetControlsDemo Properties.
//    a. On the Application tab, ensure that the Target framework is
//       .NET Framework 3.0 or later.
//    b. On the Debug tab, select Start external program and type the
//       pathname of your SOLIDWORKS Design executable.
// 5. Save the project.
// 6. In the Solution Explorer, add the following .NET Framework references:
//    * PresentationCore
//    * PresentationFramework
//    * UIAutomationProvider
//    * WindowsBase
//    * WindowsFormsIntegration
//    * System.Xaml
// 7. Copy and paste:
//    a. this into SwAddin.cs.
//    b. this into UserPMPPage.cs.
//    c. this into PMPHandler.cs.
// 8. Modify install_dir in SwAddin::WinFormInFeatureMgr to point to your
//    SOLIDWORKS Design install directory.
// 9. Add a Windows Presentation Foundation User Control to the project:
//    a. In the Solution Explorer, right-click the project and click
//       Add > User Control.
//    b. Type WPFFControl.xaml in Name.
//    c. Click the User Control (WPF) template.
//    d. Click Add.
//    e. Copy and paste this into WPFFControl.xaml.
//    f. Expand WPFFControl.xaml in the Solution Explorer and copy
//       and paste this into WPFFControl.xaml.cs.
// 10. Add a Windows Forms User Control to the project:
//    a. In the Solution Explorer, right-click the project and click
//       Add > User Control.
//    b. Click Add.
//    c. Copy and paste this into UserControl1.cs.
//    d. Expand UserControl1.cs in the Solution Explorer and copy
//       and paste this into UserControl1.Designer.cs.
// 11. Create a Windows Form:
//    a. In the Solution Explorer, right-click the project and select
//       Add > Windows Form.
//    b. Click Add.
//    c. Copy and paste this into Form1.cs.
//    d. Expand Form1.cs in the Solution Explorer and copy
//       and paste this into Form1.Designer.cs.
// 12. Save, clean, and build the project.
//
// Postconditions:
// 1. In SOLIDWORKS Design, click Tools > DotNetControlsDemo > WinFormInTaskPane.
// 2. Displays a new tab in the Task Pane.
// 3. Open a model document.
// 4. Click Tools > DotNetControlsDemo > UserControlInModelView.
// 5. Displays a .NET User Control1 tab below the model view.
// 6. Click Tools > DotNetControlsDemo > WPFFInModelView.
// 7. Displays a .NET WPF Control tab below the model view.
// 8. Click Tools > DotNetControlsDemo > WinFormInFeatureMgr.
// 9. Displays a new view in a split panel of the FeatureManager design tree.
// 10. Click Tools > DotNetControlsDemo > Show PMP.
// 11. Click OK in each of the three message boxes.
// 12. Displays a new tab with .NET controls in the PropertyManager.
//-----

//SwAddin.cs
using System;
using System.Runtime.InteropServices;
using System.Collections;
using System.Reflection;
using System.Diagnostics;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using SolidWorksTools;
using SolidWorksTools.File;
using System.Windows.Forms;
using System.Windows.Forms.Integration;

namespace DotNetControlsDemo
{
    /// <summary>
    /// Summary description for DotNetControlsDemo.

```

```

/// </summary>
[Guid("4ca45de8-6831-4a4c-83ac-d9d968803794"), ComVisible(true)]
[SwAddin(
    Description = "DotNetControlsDemo description",
    Title = "DotNetControlsDemo",
    LoadAtStartup = true
)]
public class SwAddin : ISwAddin
{
    #region Local Variables
    ISldWorks iSwApp;
    ICommandManager iCmdMgr;
    ICommandGroup cmdGroup;
    int addinID;
    Form1 TaskPanWinFormControl;
    UserControl1 TaskPanUserControl;
    Form1 FeatureMgrControl;
    UserControl1 ModelView1Control;
    WPFControl WpfModelView1Control;
    ElementHost ModelView1Host;

    #region Event Handler Variables
    Hashtable openDocs;
    SolidWorks.Interop.sldworks.SldWorks SwEventPtr;
    #endregion

    #region Property Manager Variables
    UserPMPPage ppage;
    #endregion

    // Public Properties
    public ISldWorks SwApp
    {
        get { return iSwApp; }
    }
    public ICommandManager CmdMgr
    {
        get { return iCmdMgr; }
    }

    public Hashtable OpenDocs
    {
        get { return openDocs; }
    }

    #endregion

    #region SOLIDWORKS Registration
    [ComRegisterFunctionAttribute]
    public static void RegisterFunction(Type t)
    {
        #region Get Custom Attribute: SwAddinAttribute
        SwAddinAttribute SwAttr = null;
        Type type = typeof(SwAddin);
        foreach (System.Attribute attr in type.GetCustomAttributes(false))
        {
            if (attr is SwAddinAttribute)
            {
                SwAttr = attr as SwAddinAttribute;
                break;
            }
        }
        #endregion

        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\" + t.GUID.ToString() + "}";
        Microsoft.Win32.RegistryKey addinkey = hklm.CreateSubKey(keyname);
        addinkey.SetValue(null, 0);

        addinkey.SetValue("Description", SwAttr.Description);
        addinkey.SetValue("Title", SwAttr.Title);

        keyname = "Software\\SOLIDWORKS\\AddInsStartup\\" + t.GUID.ToString() + "}";
        addinkey = hkcu.CreateSubKey(keyname);
        addinkey.SetValue(null, Convert.ToInt32(SwAttr.LoadAtStartup), Microsoft.Win32.RegistryValueKind.DWord);
    }

    [ComUnregisterFunctionAttribute]
    public static void UnregisterFunction(Type t)
    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\" + t.GUID.ToString() + "}";
        hklm.DeleteSubKey(keyname);

        keyname = "Software\\SOLIDWORKS\\AddInsStartup\\" + t.GUID.ToString() + "}";
        hkcu.DeleteSubKey(keyname);
    }

    #endregion
}

```

```

#region ISwAddin Implementation
public SwAddin()
{
}

public bool ConnectToSW(object ThisSW, int cookie)
{
    iSwApp = (ISldWorks)ThisSW;
    addinID = cookie;

    //Set up callbacks
    iSwApp.SetAddinCallbackInfo(0, this, addinID);

    #region Setup the Command Manager
    iCmdMgr = iSwApp.GetCommandManager(cookie);
    AddCommandMgr();
    #endregion

    #region Setup the Event Handlers
    SwEventPtr = (SolidWorks.Interop.sldworks.SldWorks)iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();
    #endregion

    #region Setup Sample Property Manager
    AddPMP();
    #endregion

    return true;
}

public bool DisconnectFromSW()
{
    RemoveCommandMgr();
    RemovePMP();
    DetachEventHandlers();

    iSwApp = null;
    //The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect();
    return true;
}
#endregion

#region UI Methods
public void AddCommandMgr()
{
    BitmapHandler iBmp = new BitmapHandler();

    Assembly thisAssembly;
    int cmdIndex0, cmdIndex1, cmdIndex2, cmdIndex3, cmdIndex4;
    string Title = "DotNetControlsDemo", Tooltip = "DotNetControlsDemo";

    int[] docTypes = new int[]{(int)swDocumentTypes_e.swDocASSEMBLY,
                                (int)swDocumentTypes_e.swDocDRAWING,
                                (int)swDocumentTypes_e.swDocPART};

    thisAssembly = System.Reflection.Assembly.GetAssembly(this.GetType());

    cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, Tooltip, "", -1);
    cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("DotNetControlsDemo.ToolbarLarge.bmp", thisAssembly);
    cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("DotNetControlsDemo.ToolbarSmall.bmp", thisAssembly);
    cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("DotNetControlsDemo.MainIconLarge.bmp", thisAssembly);
    cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("DotNetControlsDemo.MainIconSmall.bmp", thisAssembly);

    cmdIndex0 = cmdGroup.AddCommandItem("WinFormInTaskPane", -1, "Add Windows Form In Task Pane", "WinFormInTaskPane", 0,
    "WinFormInTaskPane", "EnableWinFormInTaskPane", 0);
    cmdIndex1 = cmdGroup.AddCommandItem("UserControlInModelView", -1, "Add User Control In Model View",
    "UserControlInModelView", 1, "UserControlInModelView", "EnableUserControlInModelView", 1);
    cmdIndex2 = cmdGroup.AddCommandItem("WPFInModelView", -1, "Add WPF In ModelView", "WPFInModelView", 2,
    "WPFInModelView", "EnableWPFInModelView", 2);
    cmdIndex3 = cmdGroup.AddCommandItem("WinFormInFeatureMgr", -1, "Add Windows Form In FeatureManager design tree",
    "WinFormInFeatureMgr", 3, "WinFormInFeatureMgr", "EnableWinFormInFeatureMgr", 3);
    cmdIndex4 = cmdGroup.AddCommandItem("Show PMP", -1, "Display PropertyManager page with .NET Controls", "Show PMP", 4,
    "ShowPMP", "EnablePMP", 4);

    cmdGroup.HasToolbar = true;
    cmdGroup.HasMenu = true;
    cmdGroup.Activate();

    bool bResult;

    foreach (int type in docTypes)
    {
        ICommandTab cmdTab;

        cmdTab = iCmdMgr.GetCommandTab(type, Title);

        if (cmdTab == null)
        {
            cmdTab = (ICommandTab)iCmdMgr.AddCommandTab(type, Title);

            CommandTabBox cmdBox = cmdTab.AddCommandTabBox();

```

```

        int[] cmdIDs = new int[6];
        int[] TextType = new int[6];

        cmdIDs[0] = cmdGroup.get_CommandID(cmdIndex0);
        TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

        cmdIDs[1] = cmdGroup.get_CommandID(cmdIndex1);
        TextType[1] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

        cmdIDs[2] = cmdGroup.get_CommandID(cmdIndex2);
        TextType[2] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

        cmdIDs[3] = cmdGroup.get_CommandID(cmdIndex3);
        TextType[3] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

        cmdIDs[4] = cmdGroup.get_CommandID(cmdIndex4);
        TextType[4] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

        cmdIDs[5] = cmdGroup.ToolbarId;
        TextType[5] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal |
        (int)swCommandTabButtonFlyoutStyle_e.swCommandTabButton_ActionFlyout;

        bResult = cmdBox.AddCommands(cmdIDs, TextType);

        CommandTabBox cmdBox1 = cmdTab.AddCommandTabBox();
        cmdIDs = new int[1];
        TextType = new int[1];

        cmdIDs[0] = cmdGroup.ToolbarId;
        TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow |
        (int)swCommandTabButtonFlyoutStyle_e.swCommandTabButton_ActionFlyout;

        bResult = cmdBox1.AddCommands(cmdIDs, TextType);

        cmdTab.AddSeparator(cmdBox1, cmdGroup.ToolbarId);
    }
}
thisAssembly = null;
iBmp.Dispose();
}

public void RemoveCommandMgr()
{
    iCmdMgr.RemoveCommandGroup(1);
}

public Boolean AddPMP()
{
    ppage = new UserPMPPage(this);
    return true;
}

public Boolean RemovePMP()
{
    ppage = null;
    return true;
}

#endregion

#region UI Callbacks
public void WinFormInTaskPane()
{
    ITaskpaneView pTaskPanView;
    pTaskPanView = iSwApp.CreateTaskpaneView2("", ".NET Windows Form Control");
    TaskPanWinFormControl = new Form1();
    pTaskPanView.DisplayWindowFromHandle64(TaskPanWinFormControl.Handle.ToInt64());
}

public int EnableWinFromInTaskPane()
{
    return 1;
}

public void UserControlInModelView()
{
    IModelDoc2 pDoc;
    pDoc = (IModelDoc2)iSwApp.ActiveDoc;
    IModelViewManager swModelViewMgr;
    swModelViewMgr = pDoc.ModelViewManager;
    ModelView1Control = new UserControl1();
    swModelViewMgr.DisplayWindowFromHandle64(".NET User Control1", ModelView1Control.Handle.ToInt64(), false);
}

public int EnableUserControlInModelView()
{
    if (iSwApp.ActiveDoc != null)
        return 1;
    else

```

```

        return 0;
    }

    public void WPFInModelView()
    {
        IModelDoc2 pDoc;
        pDoc = (IModelDoc2)iSwApp.ActiveDoc;
        IModelViewManager swModelViewMgr;
        swModelViewMgr = pDoc.ModelViewManager;
        WpfModelView1Control = new WPFControl();

        ModelView1Host = new ElementHost();
        ModelView1Host.Child = WpfModelView1Control;
        swModelViewMgr.DisplayWindowFromHandle64(".NET WPF Control", ModelView1Host.Handle.ToInt64(), true);
    }

    public int EnableWPFInModelView()
    {
        if (iSwApp.ActiveDoc != null)
            return 1;
        else
            return 0;
    }

    public void WinFormInFeatureMgr()
    {
        IModelDoc2 pDoc;
        pDoc = (IModelDoc2)iSwApp.ActiveDoc;
        IModelViewManager swModelViewMgr;
        swModelViewMgr = pDoc.ModelViewManager;
        IFeatMgrView swFeatMgrTabTop;
        FeatureMgrControl = new Form1();
        swFeatMgrTabTop =
swModelViewMgr.CreateFeatureMgrWindowFromHandle64("public_documents\\samples\\tutorial\\api\\addin\\pm_extruded_block.bmp",
FeatureMgrControl.Handle.ToInt64(), ".NET Windows Form Control", (int)swFeatMgrPane_e.swFeatMgrPaneTop);
        pDoc.FeatureManagerSplitterPosition = 0.5;
        swFeatMgrTabTop.ActivateView();
    }

    public int EnableWinFormInFeatureMgr()
    {
        if (iSwApp.ActiveDoc != null)
            return 1;
        else
            return 0;
    }

    public void ShowPMP()
    {
        if (ppage != null)
            ppage.Show();
    }

    public int EnablePMP()
    {
        if (iSwApp.ActiveDoc != null)
            return 1;
        else
            return 0;
    }
}
#endregion

#region Event Methods
public bool AttachEventHandlers()
{
    AttachSwEvents();
    //Listen for events on all currently open docs
    AttachEventsToAllDocuments();
    return true;
}

private bool AttachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify += new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 += new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 += new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify += new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify += new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

private bool DetachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify -= new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
    }
}

```

```

        SwEventPtr.DocumentLoadNotify2 -= new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 -= new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify -= new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify -= new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

public void AttachEventsToAllDocuments()
{
    ModelDoc2 modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while (modDoc != null)
    {
        if (!openDocs.Contains(modDoc))
        {
            AttachModelDocEventHandler(modDoc);
        }
        modDoc = (ModelDoc2)modDoc.GetNext();
    }
}

public bool AttachModelDocEventHandler(ModelDoc2 modDoc)
{
    if (modDoc == null)
        return false;

    DocumentEventHandler docHandler = null;

    if (!openDocs.Contains(modDoc))
    {
        switch (modDoc.GetType())
        {
            case (int)swDocumentTypes_e.swDocPART:
            {
                docHandler = new PartEventHandler(modDoc, this);
                break;
            }
            case (int)swDocumentTypes_e.swDocASSEMBLY:
            {
                docHandler = new AssemblyEventHandler(modDoc, this);
                break;
            }
            case (int)swDocumentTypes_e.swDocDRAWING:
            {
                docHandler = new DrawingEventHandler(modDoc, this);
                break;
            }
            default:
            {
                return false; //Unsupported document type
            }
        }
        docHandler.AttachEventHandlers();
        openDocs.Add(modDoc, docHandler);
    }
    return true;
}

public bool DetachModelEventHandler(ModelDoc2 modDoc)
{
    DocumentEventHandler docHandler;
    docHandler = (DocumentEventHandler)openDocs[modDoc];
    openDocs.Remove(modDoc);
    modDoc = null;
    docHandler = null;
    return true;
}

public bool DetachEventHandlers()
{
    DetachSwEvents();

    //Close events on all currently open docs
    DocumentEventHandler docHandler;
    int numKeys = openDocs.Count;
    object[] keys = new Object[numKeys];

    //Remove all document event handlers
    openDocs.Keys.CopyTo(keys, 0);
    foreach (ModelDoc2 key in keys)
    {
        docHandler = (DocumentEventHandler)openDocs[key];
        docHandler.DetachEventHandlers(); //This also removes the pair from the hash
        docHandler = null;
    }
    return true;
}
#endregion

#region Event Handlers

```

```

//Events
public int OnDocChange()
{
    return 0;
}

public int OnDocLoad(string docTitle, string docPath)
{
    return 0;
}

int FileOpenPostNotify(string FileName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int OnFileNew(object newDoc, int docType, string templateName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int OnModelChange()
{
    return 0;
}

#endregion
}
}

```

[Back to top](#)

```

//UserPMPage.cs
using System;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using System.Windows.Forms;

using System.Windows.Forms.Integration;

namespace DotNetControlsDemo
{
    public class UserPMPage
    {
        //Local Objects
        IPropertyManagerPage2 swPropertyPage;
        PMPHandler handler;
        ISldWorks iSwApp;
        SwAddin userAddin;

        WPFControl MyWPFControl;
        ElementHost elhost;
        UserControl1 MyUserControl;
        Form1 MyWinFormControl;

        #region Property Manager Page Controls
        //Groups
        IPropertyManagerPageGroup group1;
        IPropertyManagerPageGroup group2;
        IPropertyManagerPageGroup group3;

        //Controls
        IPropertyManagerPageWindowFromHandle dotnet1;
        IPropertyManagerPageWindowFromHandle dotnet2;
        IPropertyManagerPageWindowFromHandle dotnet3;

        //Control IDs
        public const int group1ID = 0;
        public const int group2ID = 1;
        public const int group3ID = 2;

        public const int DotNet1ID = 3;
        public const int DotNet2ID = 4;
        public const int DotNet3ID = 5;

        #endregion

        public UserPMPage(SwAddin addin)
        {
            userAddin = addin;
            iSwApp = (ISldWorks)userAddin.SwApp;
            CreatePropertyManagerPage();
        }

        protected void CreatePropertyManagerPage()
        {
            int errors = -1;
            int options = (int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_OkayButton |
                (int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_CancelButton;

            handler = new PMPHandler(userAddin);
        }
    }
}

```

```

errors);
        swPropertyPage = (IPropertyManagerPage2)iSwApp.CreatePropertyManagerPage(".Net PMP control", options, handler, ref
    if (swPropertyPage != null && errors == (int)swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay)
    {
        try
        {
            AddControls();
        }
        catch (Exception e)
        {
            iSwApp.SendMsgToUser2(e.Message, 0, 0);
        }
    }
}

//Controls display on the page top to bottom in the order
//in which they are added to the page
protected void AddControls()
{
    short controlType = -1;
    short align = -1;
    int options = -1;

    //Add the groups
    options = (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded |
        (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Visible;

    group1 = (IPropertyManagerPageGroup)swPropertyPage.AddGroupBox(group1ID, "Windows Form controls", options);
    options = (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Checkbox |
        (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Visible;

    group2 = (IPropertyManagerPageGroup)swPropertyPage.AddGroupBox(group2ID, "User controls", options);
    options = (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Checkbox |
        (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Visible;

    group3 = (IPropertyManagerPageGroup)swPropertyPage.AddGroupBox(group3ID, "WPF controls", options);

    controlType = (int)swPropertyManagerPageControlType_e.swControlType_WindowFromHandle;
    align = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge;
    options = (int)swAddControlOptions_e.swControlOptions_Enabled |
        (int)swAddControlOptions_e.swControlOptions_Visible;

    dotnet1 = (IPropertyManagerPageWindowFromHandle)group1.AddControl2(DotNet1ID, controlType, ".Net Windows Form
control", align, options, "This control is added without COM");
    dotnet1.Height = 150;

    controlType = (int)swPropertyManagerPageControlType_e.swControlType_WindowFromHandle;
    align = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge;
    options = (int)swAddControlOptions_e.swControlOptions_Enabled |
        (int)swAddControlOptions_e.swControlOptions_Visible;

    dotnet2 = (IPropertyManagerPageWindowFromHandle)group2.AddControl2(DotNet2ID, controlType, ".Net user form control",
align, options, "This control is added without COM");
    dotnet2.Height = 150;

    dotnet3 = (IPropertyManagerPageWindowFromHandle)group3.AddControl2(DotNet3ID, controlType, ".Net WPF control", align,
options, "This control is added without COM");
    dotnet3.Height = 50;

}

public void Show()
{
    //Windows Form control
    MyWinFormControl = new Form1();
    //If you are adding Windows form in the PropertyManager Page, set TopLevel to false
    MyWinFormControl.TopLevel = false;
    MyWinFormControl.Show();
    dotnet1.SetWindowHandle64(MyWinFormControl.Handle.ToInt64());

    //User control
    MyUserControl = new UserControl1();
    dotnet2.SetWindowHandle64(MyUserControl.Handle.ToInt64());

    //WPF control
    elhost = new ElementHost();
    MyWPFControl = new WPFControl();
    elhost.Child = MyWPFControl;
    dotnet3.SetWindowHandle64(elhost.Handle.ToInt64());

    //Show PropertyManager page
    swPropertyPage.Show();
}
}
}

```

```
//PMPHandler.cs
using System;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using System.Windows.Forms;

namespace DotNetControlsDemo
{
    public class PMPHandler : IPropertyManagerPage2Handler9
    {
        ISldWorks iSwApp;
        SwAddin userAddin;

        public PMPHandler(SwAddin addin)
        {
            userAddin = addin;
            iSwApp = (ISldWorks)userAddin.SwApp;
        }

        //Implement these methods from the interface
        public void AfterClose()
        {
            //This function must contain code, even if it does nothing, to prevent the
            //NET runtime environment from doing garbage collection at the wrong time.
            int IndentSize;
            IndentSize = System.Diagnostics.Debug.IndentSize;
            System.Diagnostics.Debug.WriteLine(IndentSize);
        }

        public void OnCheckboxCheck(int id, bool status)
        {
        }

        public void OnClose(int reason)
        {
            //This function must contain code, even if it does nothing, to prevent the
            //NET runtime environment from doing garbage collection at the wrong time.
            int IndentSize;
            IndentSize = System.Diagnostics.Debug.IndentSize;
            System.Diagnostics.Debug.WriteLine(IndentSize);
        }

        public void OnComboboxEditChanged(int id, string text)
        {
        }

        public int OnActiveXControlCreated(int id, bool status)
        {
            return -1;
        }

        public void OnButtonPress(int id)
        {
        }

        public void OnComboboxSelectionChanged(int id, int item)
        {
        }

        public void OnGroupCheck(int id, bool status)
        {
        }

        public void OnGroupExpand(int id, bool status)
        {
        }

        public bool OnHelp()
        {
            return true;
        }

        public void OnListboxSelectionChanged(int id, int item)
        {
        }

        public bool OnNextPage()
        {
            return true;
        }

        public void OnNumberboxChanged(int id, double val)
        {
        }
    }
}
```

```

}

public void OnOptionCheck(int id)
{
}

public bool OnPreviousPage()
{
    return true;
}

public void OnSelectionboxCalloutCreated(int id)
{
}

public void OnSelectionboxCalloutDestroyed(int id)
{
}

public void OnSelectionboxFocusChanged(int id)
{
}

public void OnSelectionboxListChanged(int id, int item)
{
}

public void OnTextboxChanged(int id, string text)
{
}

public void AfterActivation()
{
}

public bool OnKeystroke(int Wparam, int Message, int Lparam, int Id)
{
    return true;
}

public void OnPopupMenuItem(int Id)
{
}

public void OnPopupMenuItemUpdate(int Id, ref int retval)
{
}

public bool OnPreview()
{
    return true;
}

public void OnSliderPositionChanged(int Id, double Value)
{
}

public void OnSliderTrackingCompleted(int Id, double Value)
{
}

public bool OnSubmitSelection(int Id, object Selection, int SelType, ref string ItemText)
{
    return true;
}

public bool OnTabClicked(int Id)
{
    return true;
}

public void OnUndo()
{
}

public void OnWhatsNew()
{
}

public void OnRedo()
{
}

```

```

        public void OnGainedFocus(int Id)
        {
        }

        public void OnLostFocus(int Id)
        {
        }

        public int OnWindowFromHandleControlCreated(int id, bool status)
        {
            System.Windows.Forms.MessageBox.Show(".NET control created");
            return -1;
        }

        public void OnListBoxRMBUp(int Id, int PosX, int PosY)
        {
        }

        public void OnNumberBoxTrackingCompleted(int Id, double Value)
        {
        }
    }
}

```

[Back to top](#)

```

//Form1.cs
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace DotNetControlsDemo
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {
        }

        private void textBox1_TextChanged(object sender, EventArgs e)
        {
        }

        private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
        {
        }

        private void listBox1_SelectedIndexChanged(object sender, EventArgs e)
        {
        }
    }
}

```

[Back to top](#)

```

//Form1.Designer.cs
namespace DotNetControlsDemo
{
    partial class Form1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
        }
    }
}

```

```

{
    if (disposing && (components != null))
    {
        components.Dispose();
    }
    base.Dispose(disposing);
}

#region Windows Form Designer generated code

/// <summary>
/// Required method for Designer support - do not modify
/// the contents of this method with the code editor.
/// </summary>
private void InitializeComponent()
{
    this.listBox1 = new System.Windows.Forms.ListBox();
    this.comboBox1 = new System.Windows.Forms.ComboBox();
    this.textBox1 = new System.Windows.Forms.TextBox();
    this.button1 = new System.Windows.Forms.Button();
    this.SuspendLayout();
    //
    // listBox1
    //
    this.listBox1.Anchor = ((System.Windows.Forms.AnchorStyles)((((System.Windows.Forms.AnchorStyles.Bottom |
System.Windows.Forms.AnchorStyles.Left)
| System.Windows.Forms.AnchorStyles.Right)));
    this.listBox1.FormattingEnabled = true;
    this.listBox1.Items.AddRange(new object[] {
    "Help",
    "Whats New"});
    this.listBox1.Location = new System.Drawing.Point(25, 160);
    this.listBox1.Name = "listBox1";
    this.listBox1.Size = new System.Drawing.Size(304, 43);
    this.listBox1.TabIndex = 10;
    this.listBox1.SelectedIndexChanged += new System.EventHandler(this.listBox1_SelectedIndexChanged);
    //
    // comboBox1
    //
    this.comboBox1.DisplayMember = "Monday";
    this.comboBox1.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
    this.comboBox1.FormattingEnabled = true;
    this.comboBox1.Items.AddRange(new object[] {
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday"});
    this.comboBox1.Location = new System.Drawing.Point(25, 122);
    this.comboBox1.Name = "comboBox1";
    this.comboBox1.Size = new System.Drawing.Size(140, 21);
    this.comboBox1.TabIndex = 9;
    this.comboBox1.ValueMember = "Monday";
    this.comboBox1.SelectedIndexChanged += new System.EventHandler(this.comboBox1_SelectedIndexChanged);
    //
    // textBox1
    //
    this.textBox1.Anchor = ((System.Windows.Forms.AnchorStyles)((((System.Windows.Forms.AnchorStyles.Top |
System.Windows.Forms.AnchorStyles.Bottom)
| System.Windows.Forms.AnchorStyles.Left)
| System.Windows.Forms.AnchorStyles.Right)));
    this.textBox1.Location = new System.Drawing.Point(25, 86);
    this.textBox1.Name = "textBox1";
    this.textBox1.Size = new System.Drawing.Size(321, 20);
    this.textBox1.TabIndex = 8;
    this.textBox1.TextChanged += new System.EventHandler(this.textBox1_TextChanged);
    //
    // button1
    //
    this.button1.Location = new System.Drawing.Point(25, 12);
    this.button1.Name = "button1";
    this.button1.Size = new System.Drawing.Size(140, 57);
    this.button1.TabIndex = 7;
    this.button1.Text = "Say Hello";
    this.button1.UseVisualStyleBackColor = true;
    this.button1.Click += new System.EventHandler(this.button1_Click);
    //
    // Form1
    //
    this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
    this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
    this.ClientSize = new System.Drawing.Size(382, 215);
    this.ControlBox = false;
    this.Controls.Add(this.listBox1);
    this.Controls.Add(this.comboBox1);
    this.Controls.Add(this.textBox1);
    this.Controls.Add(this.button1);
    this.FormBorderStyle = System.Windows.Forms.FormBorderStyle.None;
    this.MaximizeBox = false;
    this.MinimizeBox = false;
    this.Name = "Form1";
    this.ShowIcon = false;
    this.ShowInTaskbar = false;
    this.Text = "Form1";
}

```

```

        this.ResumeLayout(false);
        this.PerformLayout();
    }

    #endregion

    private System.Windows.Forms.ListBox listBox1;
    private System.Windows.Forms.ComboBox comboBox1;
    private System.Windows.Forms.TextBox textBox1;
    private System.Windows.Forms.Button button1;
}
}

```

[Back to top](#)

```

//UserController1.cs
using System;
using System.Drawing;
using System.Data;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace DotNetControlsDemo
{
    public partial class UserControl1 : UserControl
    {
        public UserControl1()
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {
        }

        private void textBox1_TextChanged(object sender, EventArgs e)
        {
        }

        private void radioButton1_CheckedChanged(object sender, EventArgs e)
        {
        }

        private void tabPage1_Click(object sender, EventArgs e)
        {
        }

        private void tabPage2_Click(object sender, EventArgs e)
        {
        }

        private void tabPage3_Click(object sender, EventArgs e)
        {
        }
    }
}

```

[Back to top](#)

```

//UserController1.Designer.cs
namespace DotNetControlsDemo
{
    partial class UserControl1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Component Designer generated code

```

```

/// <summary>
/// Required method for Designer support - do not modify
/// the contents of this method with the code editor.
/// </summary>
private void InitializeComponent()
{
    this.radioButton1 = new System.Windows.Forms.RadioButton();
    this.textBox1 = new System.Windows.Forms.TextBox();
    this.button1 = new System.Windows.Forms.Button();
    this.tabControl1 = new System.Windows.Forms.TabControl();
    this.TabPage1 = new System.Windows.Forms.TabPage();
    this.TabPage2 = new System.Windows.Forms.TabPage();
    this.TabPage3 = new System.Windows.Forms.TabPage();
    this.tabControl1.SuspendLayout();
    this.SuspendLayout();
    //
    // radioButton1
    //
    this.radioButton1.Anchor = System.Windows.Forms.AnchorStyles.Left;
    this.radioButton1.AutoSize = true;
    this.radioButton1.Location = new System.Drawing.Point(23, 113);
    this.radioButton1.Name = "radioButton1";
    this.radioButton1.Size = new System.Drawing.Size(85, 17);
    this.radioButton1.TabIndex = 5;
    this.radioButton1.TabStop = true;
    this.radioButton1.Text = "radioButton1";
    this.radioButton1.UseVisualStyleBackColor = true;
    this.radioButton1.CheckedChanged += new System.EventHandler(this.radioButton1_CheckedChanged);
    //
    // textBox1
    //
    this.textBox1.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Top |
System.Windows.Forms.AnchorStyles.Left |
System.Windows.Forms.AnchorStyles.Right)));
    this.textBox1.Location = new System.Drawing.Point(23, 87);
    this.textBox1.Name = "textBox1";
    this.textBox1.Size = new System.Drawing.Size(378, 20);
    this.textBox1.TabIndex = 4;
    this.textBox1.TextChanged += new System.EventHandler(this.textBox1_TextChanged);
    //
    // button1
    //
    this.button1.Location = new System.Drawing.Point(23, 17);
    this.button1.Name = "button1";
    this.button1.Size = new System.Drawing.Size(119, 49);
    this.button1.TabIndex = 3;
    this.button1.Text = "Say Hello";
    this.button1.UseVisualStyleBackColor = true;
    this.button1.Click += new System.EventHandler(this.button1_Click);
    //
    // tabControl1
    //
    this.tabControl1.Anchor = ((System.Windows.Forms.AnchorStyles)((System.Windows.Forms.AnchorStyles.Bottom |
System.Windows.Forms.AnchorStyles.Left |
System.Windows.Forms.AnchorStyles.Right)));
    this.tabControl1.Controls.Add(this.TabPage1);
    this.tabControl1.Controls.Add(this.TabPage2);
    this.tabControl1.Controls.Add(this.TabPage3);
    this.tabControl1.Location = new System.Drawing.Point(23, 148);
    this.tabControl1.Multiline = true;
    this.tabControl1.Name = "tabControl1";
    this.tabControl1.SelectedIndex = 0;
    this.tabControl1.Size = new System.Drawing.Size(386, 69);
    this.tabControl1.TabIndex = 6;
    //
    // TabPage1
    //
    this.TabPage1.Location = new System.Drawing.Point(4, 22);
    this.TabPage1.Name = "TabPage1";
    this.TabPage1.Padding = new System.Windows.Forms.Padding(3);
    this.TabPage1.Size = new System.Drawing.Size(378, 43);
    this.TabPage1.TabIndex = 0;
    this.TabPage1.Text = "TabPage1";
    this.TabPage1.UseVisualStyleBackColor = true;
    this.TabPage1.Click += new System.EventHandler(this.TabPage1_Click);
    //
    // TabPage2
    //
    this.TabPage2.Location = new System.Drawing.Point(4, 22);
    this.TabPage2.Name = "TabPage2";
    this.TabPage2.Padding = new System.Windows.Forms.Padding(3);
    this.TabPage2.Size = new System.Drawing.Size(378, 43);
    this.TabPage2.TabIndex = 1;
    this.TabPage2.Text = "TabPage2";
    this.TabPage2.UseVisualStyleBackColor = true;
    this.TabPage2.Click += new System.EventHandler(this.TabPage2_Click);
    //
    // TabPage3
    //
    this.TabPage3.Location = new System.Drawing.Point(4, 22);
    this.TabPage3.Name = "TabPage3";
    this.TabPage3.Padding = new System.Windows.Forms.Padding(3);
    this.TabPage3.Size = new System.Drawing.Size(378, 43);
    this.TabPage3.TabIndex = 2;
    this.TabPage3.Text = "TabPage3";

```

```

        this.TabPage3.UseVisualStyleBackColor = true;
        this.TabPage3.Click += new System.EventHandler(this.TabPage3_Click);
        //
        // UserControl1
        //
        this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
        this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
        this.Controls.Add(this.tabControl1);
        this.Controls.Add(this.radioButton1);
        this.Controls.Add(this.textBox1);
        this.Controls.Add(this.button1);
        this.Name = "UserControl1";
        this.Size = new System.Drawing.Size(427, 237);
        this.tabControl1.ResumeLayout(false);
        this.ResumeLayout(false);
        this.PerformLayout();
    }

    #endregion

    private System.Windows.Forms.RadioButton radioButton1;
    private System.Windows.Forms.TextBox textBox1;
    private System.Windows.Forms.Button button1;
    private System.Windows.Forms.TabControl tabControl1;
    private System.Windows.Forms.TabPage tabPage1;
    private System.Windows.Forms.TabPage tabPage2;
    private System.Windows.Forms.TabPage tabPage3;
}

```

[Back to top](#)

```

//WPFCtrl.xaml
<UserControl x:Class="DotNetControlsDemo.WPFCtrl"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Height="204" Width="314" Loaded="UserControl_Loaded">
    <Grid Height="44" Width="108">
        <ComboBox Text="WPF" Height="44" VerticalAlignment="Top">
            <ComboBoxItem Margin="2">
                <Ellipse Width="20" Height="20" Stroke="Black" />
            </ComboBoxItem>
            <ComboBoxItem Margin="2">
                <Path Stroke="Black" Data="M 0,0 L 20,0 L 10,20 Z" />
            </ComboBoxItem>
            <ComboBoxItem Margin="2">
                <Rectangle Stroke="Black" Width="20" Height="20" />
            </ComboBoxItem>
        </ComboBox>
    </Grid>
</UserControl>

```

[Back to top](#)

```

//WPFCtrl.xaml.cs
using System;
using System.Collections.Generic;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace DotNetControlsDemo
{
    /// <summary>
    /// Interaction logic for WPFCtrl.xaml
    /// </summary>
    public partial class WPFCtrl : UserControl
    {
        public WPFCtrl()
        {
            InitializeComponent();
        }

        private void UserControl_Loaded(object sender, RoutedEventArgs e)
        {
        }
    }
}

```

[Back to top](#)

